3. Process Management

Operating Systems (OS)
Master in Innovation and Research in Informatics (MIRI)
Facultat d'Informàtica de Barcelona (FIB)
Universitat Politècnica de Catalunya (UPC)
2025/2026 Q1

FIB

Contents

- **Introduction**
- Process scheduling
- Process resilience
- Virtualization

Introduction

- Process management features in a desktop O.S.
  - Creating and terminating processes
  - Suspending and resuming processes
  - Scheduling processes and threads on the CPU
  - Providing mechanisms for process synchronization
  - Providing mechanisms for process communication

Introduction

- Process management features in a WSC O.S.
  - Unchanged activities, done at local level
    - Creating and terminating processes
    - Suspending and resuming processes
  - Adapted features for the WSC architecture
    - Scheduling processes and threads on the nodes/CPU(s)
    - Providing mechanisms for process synchronization
    - Providing mechanisms for process communication
    - Supporting and exploiting full-system virtualization
  - New features required by the WSC architecture
    - Providing mechanisms for process resilience

Contents

- Introduction
- **Process scheduling**
- Process resilience
- Virtualization

Contents

- Introduction
- **Process scheduling**
  - **Introduction**
  - Scheduler architectures
  - Time-sharing
  - Space-sharing
  - PE allocation
  - Gang scheduling
  - Metrics
- Process resilience
- Virtualization

Introduction

- In a desktop OS, there is only one dimension: **which** process should be run next?
  - Various scheduling policies, for example:
  A. First Come First Served (FCFS): the first process in the ready queue, the next to run
     - Non-preemptive: process only leaves CPU voluntarily
  B. Priorities: process with highest, the next to run
     - Can be non-preemptive or preemptive
     - Priority may change over time
  C. Round Robin: each process has a time quantum
     - Can leave the CPU voluntarily or it is preempted when its quantum is over

Introduction

- In a WSC OS, there are two dimensions:
  1. **Which** process should be run next?
  2. **Where** must the process run?
     - i.e. on which Processing Element (PE): CPU, computer, computer/CPU (i.e. two-level allocation)
- Two approaches to decide the mappings over time between processes and PEs
  1. **Time-sharing:** multiple processes run on the same PEs by doing recurrent context switching
  2. **Space-sharing:** PEs are partitioned among multiple processes that execute side by side

Job definition

- A job is an application in execution (as known by the OS), composed of multiple concurrent processes/threads
- A job is the unit to schedule defined by:
  i. Arrival time
  ii. Number of processes/threads = number of required PEs (a.k.a. job width or size)
  iii. Runtime estimation (a.k.a. job length)
- Jobs can be classified in several ways

Job classification

- Based on their number of processes/threads
  a) **Serial**
  b) **Parallel**
- Based on their interaction with the user
  a) **Interactive**
    - Their input/output is directed to a terminal, so users expect them to execute immediately, not to wait in a queue. They require fast turnaround time
  b) **Batch**
    - They do not need immediate responses, can be submitted to a queue to be scheduled to run when the resources become available

Job classification

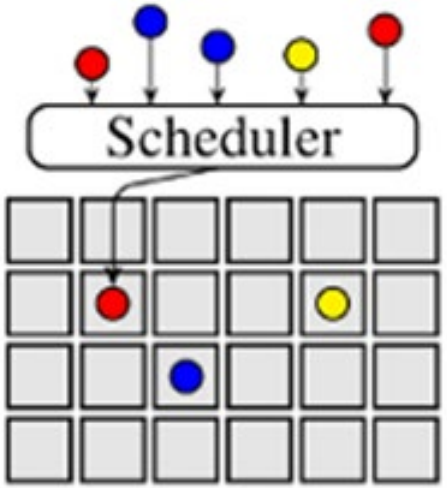
- Based on the capacity to adjust their number of processes/threads
  a) **Rigid:** Number of processes is predefined
  b) **Moldable:** Number of processes is determined at start time and it cannot change thereafter
  c) **Evolving:** Job changes its number of processes at runtime on its own initiative and requests the corresponding PEs to the scheduler
  d) **Malleable:** Job changes its number of processes at runtime according to the availability of PEs gathered from the scheduler

Contents

- Introduction
- **Process scheduling**
  - Introduction
  - **Scheduler architectures**
  - Time-sharing
  - Space-sharing
  - PE allocation
  - Gang scheduling
  - Metrics
- Process resilience
- Virtualization

Monolithic scheduler

- A single scheduler handles all the jobs using the same scheduling logic
- Accurate and up-to-date view of the entire cluster state: good scheduling decisions (↑), easy to enforce global constraints (↑), slow scheduling (↓)
  - More suitable for long running jobs



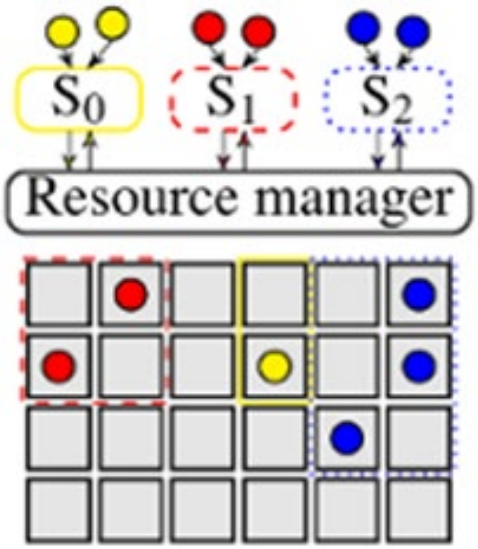
↓ Scheduler can be a single point of failure and a performance bottleneck (not very scalable)

↓ Handling of mixed workloads: intricate scheduling logic, queueing effects (head-of-line blocking)

- e.g. HPC schedulers, Google Borg, Paragon

Two-level scheduler

- A resource manager partitions the cluster dynamically among multiple independent workload-specific schedulers responsible for making the actual task placement
  - Separate concerns of resource allocation and task placement



↑ Placement logic can be tailored for specific workloads

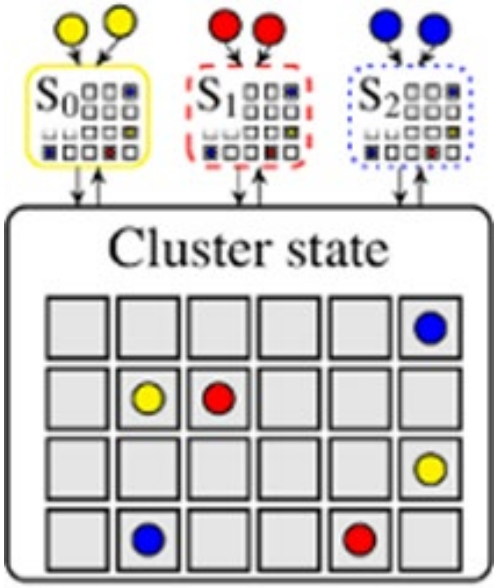
↑ Mitigate the load on the scheduler: better scalability

↓ Schedulers merely see their allocated resources: hard to enforce global constraints or detect interference

- e.g. Apache Mesos, Hadoop YARN

Shared-state scheduler

- Multiple workload-specific schedulers have access to the **entire** cluster
- Cluster state is replicated and can be independently updated by schedulers using optimistic concurrency control
  - Update may fail if there are conflicting changes and should be retried



↑ Fast scheduling and good scalability

↑ Placement logic can be tailored for specific workloads

↓ Occasionally, must decide with stale information

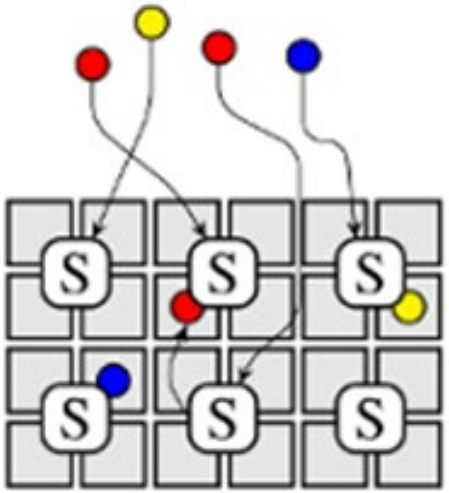
↓ Degraded performance under high contention

- e.g. Google Omega, Microsoft Apollo

Courtesy of [Schw16]

Fully-distributed scheduler

- Multiple independent schedulers service incoming workloads without any coordination between them
- They use only their local, partial, and often outdated view of cluster state to place tasks anywhere in the cluster
- Jobs can be submitted to any scheduler



↑ Fast scheduling, high throughput, and high scalability

↓ Can make bad scheduling decisions, and is hard to enforce global constraints or detect interference

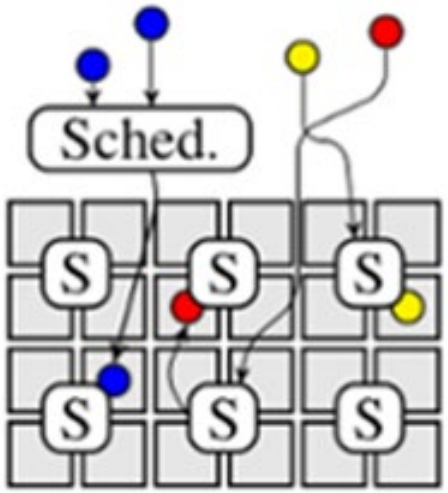
- More suitable for short running jobs and time-sensitive jobs

- e.g. Sparrow

Hybrid scheduler

- Combine fully-distributed architecture with monolithic or shared-state design
- Two scheduling paths:

1. Distributed for part of the workload (short jobs)
2. Centralized for the rest (long jobs)



- Each constituent part leverages the strengths and weaknesses discussed before

↑ Useful when there are heterogeneous workloads

  – A small number of long running (batch) jobs and a very large number of short running (low-latency) jobs

- e.g. Hawk, Mercury

READING REPORT

[Schwarzkopf13] Schwarzkopf, M., Konwinski, A., Abd-El-Malek, M., Wilkes, J., *Omega: flexible, scalable schedulers for large compute clusters*. Proceedings of the 8th ACM European Conference on Computer Systems (EuroSys'13), Prague, Czech Republic, April 14-17, 2013, pp. 351-364

Contents

- Introduction
- **Process scheduling**
  - Introduction
  - Scheduler architectures
  - **Time-sharing**
  - Space-sharing
  - PE allocation
  - Gang scheduling
  - Metrics
- Process resilience
- Virtualization

Time-sharing

- Multiple processes assigned to the same PEs
- Processes can be preempted by others during execution and restarted later

↑ Increases utilization of PEs

↓ Overhead of context switch

- Enhanced versions of scheduling policies for desktop OS are used
  - Global queue vs. local queues
- Processes are scheduled uncoordinatedly

Time-sharing: single global queue

- Put all jobs that need to be scheduled into a single global queue

↑ Simplicity

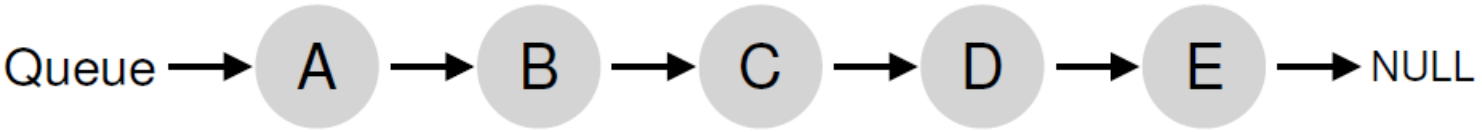
↑ Automatic load balance: workload is distributed evenly across all the PEs

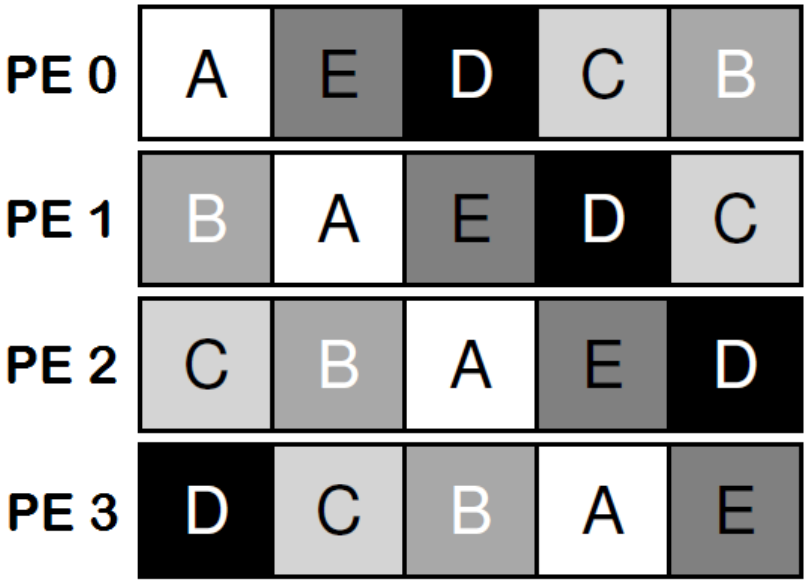
↓ Without shared memory, a global queue is hard to implement efficiently

↓ Needs synchronization to access the global queue: low scalability

↓ Does not preserve data locality

Time-sharing: single queue: example





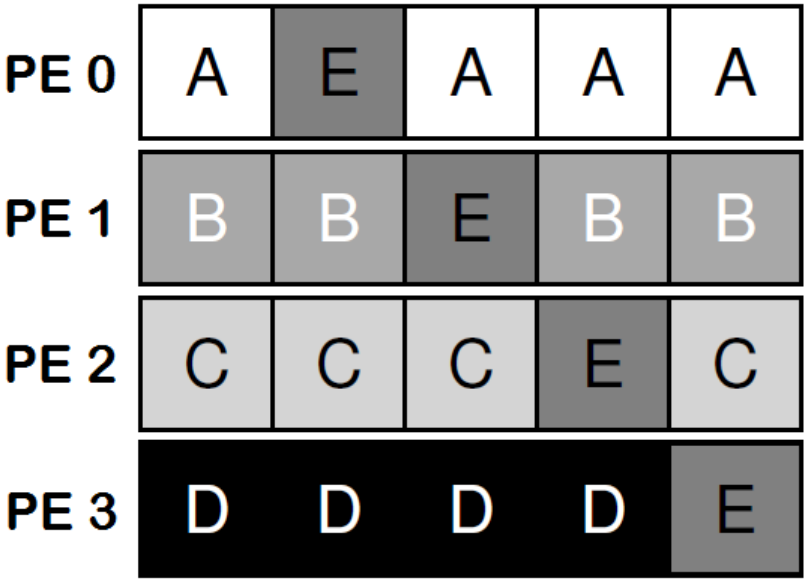
Courtesy of [ArAr23]

Time-sharing: affinity scheduling

- Schedule a process on the same PE it used before

a) **Soft affinity:** if possible

b) **Hard affinity:** necessarily



Courtesy of [ArAr23]

Time-sharing: local queues

- Independent queues for each PE
- Map processes to PEs and then schedule normally in the local queue of each PE

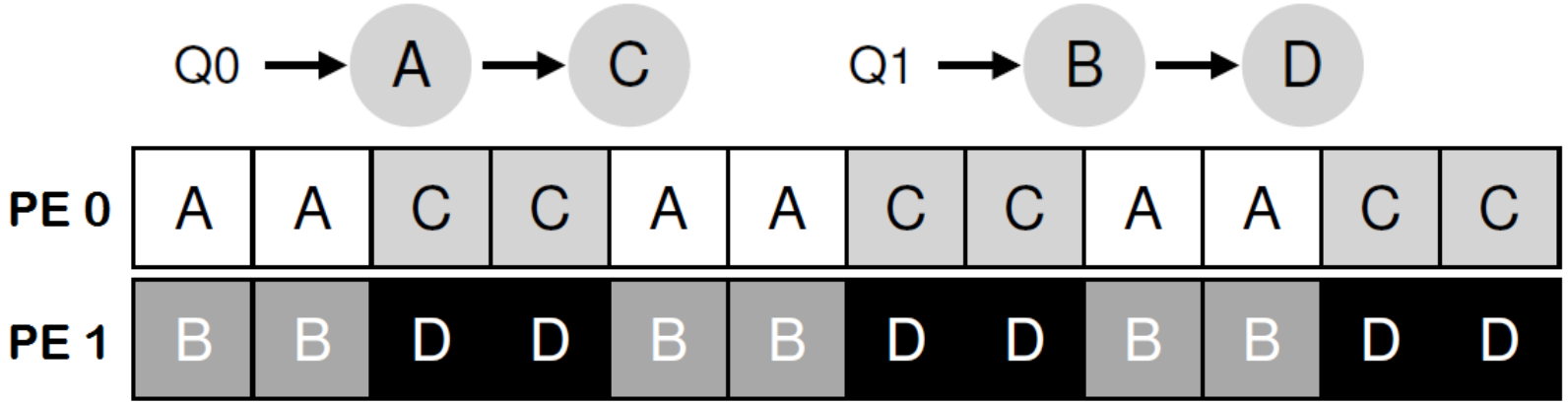
↑ Preserves data locality

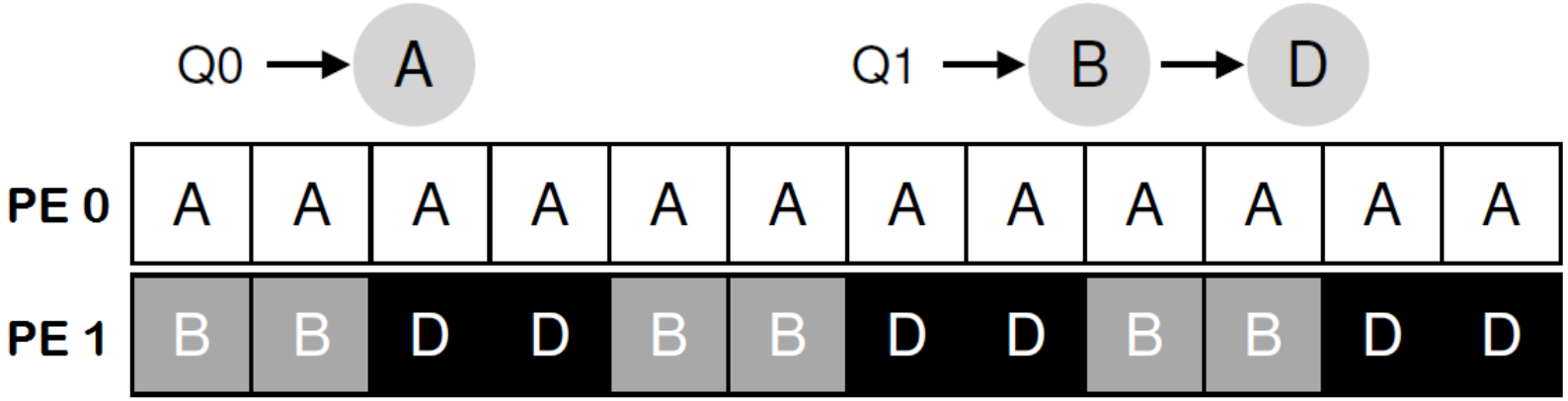
↑ Good scalability

↑ Synchronization not needed

↓ Load imbalance

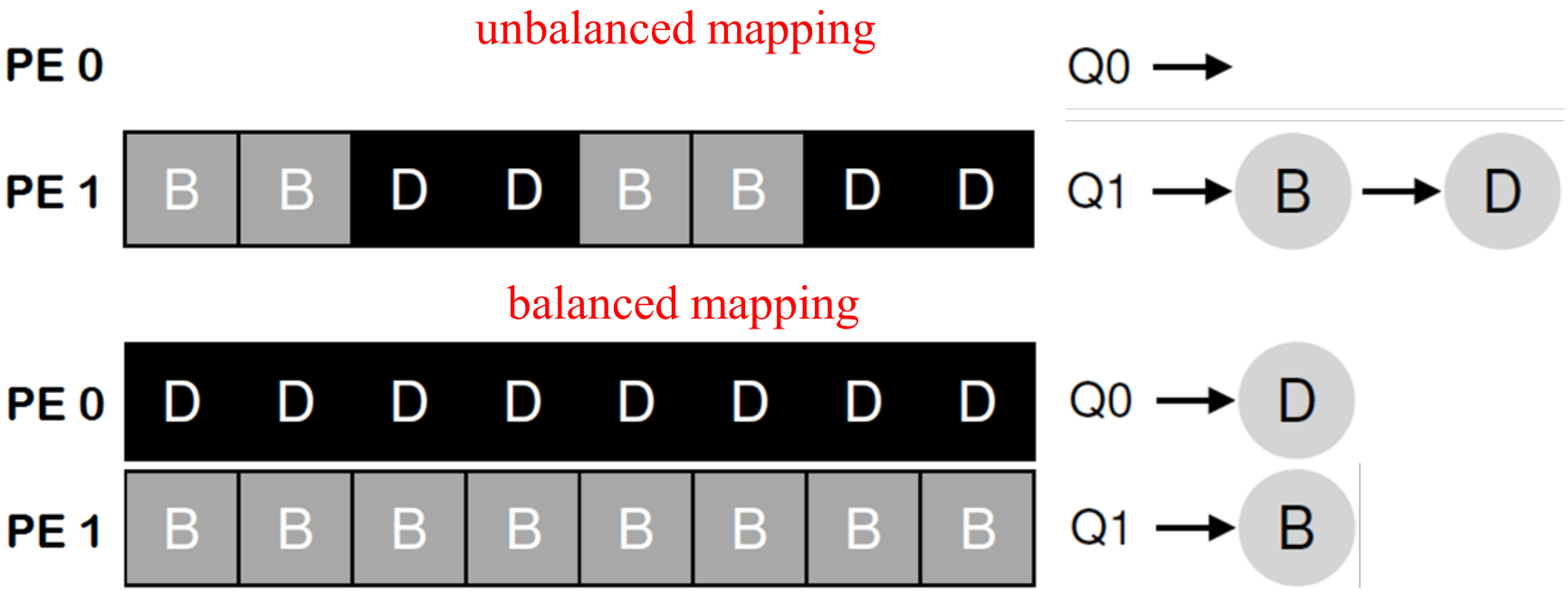
Time-sharing: local queues: example





Time-sharing: load balancing

A. Balance only when mapping new processes to PEs
   - Random, (Weighted) Round-Robin, Join-Shortest-Queue, …
   - ↑ Simplicity and low communication costs
   - ↓ Potential imbalance due to varying lengths of processes



Time-sharing: load balancing

B. Balance by migrating active processes between PEs

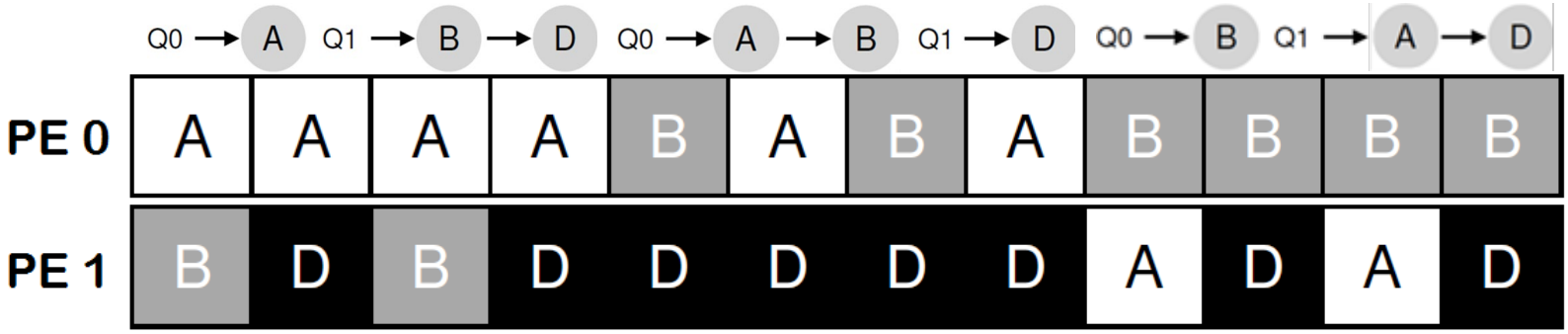
- Receiver-initiated (a.k.a. work stealing): idle PEs 'steal' processes from overloaded PEs
- Sender-initiated (a.k.a. work pushing): overloaded PEs offload processes to idle (or underloaded) PEs

↑ Better load distribution

↓ Additional communication and processing overhead

↓ Does not preserve data locality

A runs alone on a PE while B and D alternate on the other. After some time, they rotate



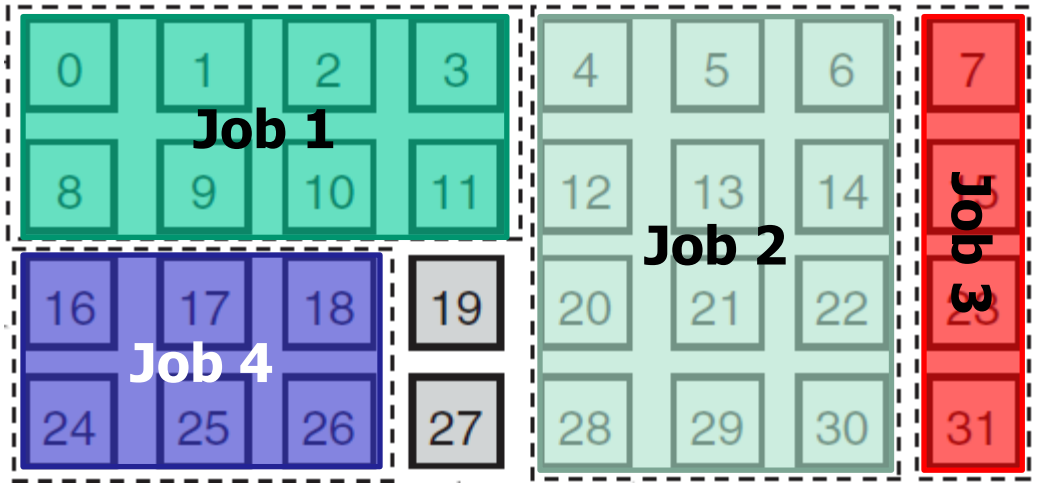
Courtesy of [ArAr23]

Contents

- Introduction
- **Process scheduling**
  - Introduction
  - Scheduler architectures
  - Time-sharing
  - **Space-sharing**
  - PE allocation
  - Gang scheduling
  - Metrics
- Process resilience
- Virtualization

Space-sharing

- Divide PEs into disjoint partitions and assign a single job to each partition
  - Often, only a process is assigned to a PE at a time
    - Alternatively, combine with time-sharing to schedule multiple processes within a PE
  - Jobs run side-by-side **until completion**



Space-sharing

- Advantages
  - ↑ Partition is dedicated to a job
  - ↑ Avoids context switching overhead
- Disadvantages
  - ↓ Interconnect and I/O system may be shared by several jobs
  - ↓ Poor utilization (PE wasted when a process blocks)
- Partitioning mechanisms classify into 4 types: i) fixed, ii) variable, iii) adaptive, iv) dynamic

Space-sharing

a) Fixed partitioning

- Partitions set by the administrator
  - Typically, through several queues with different sizes and limits (number of PEs, execution time, ...)
- Partitions can be modified only by reboot

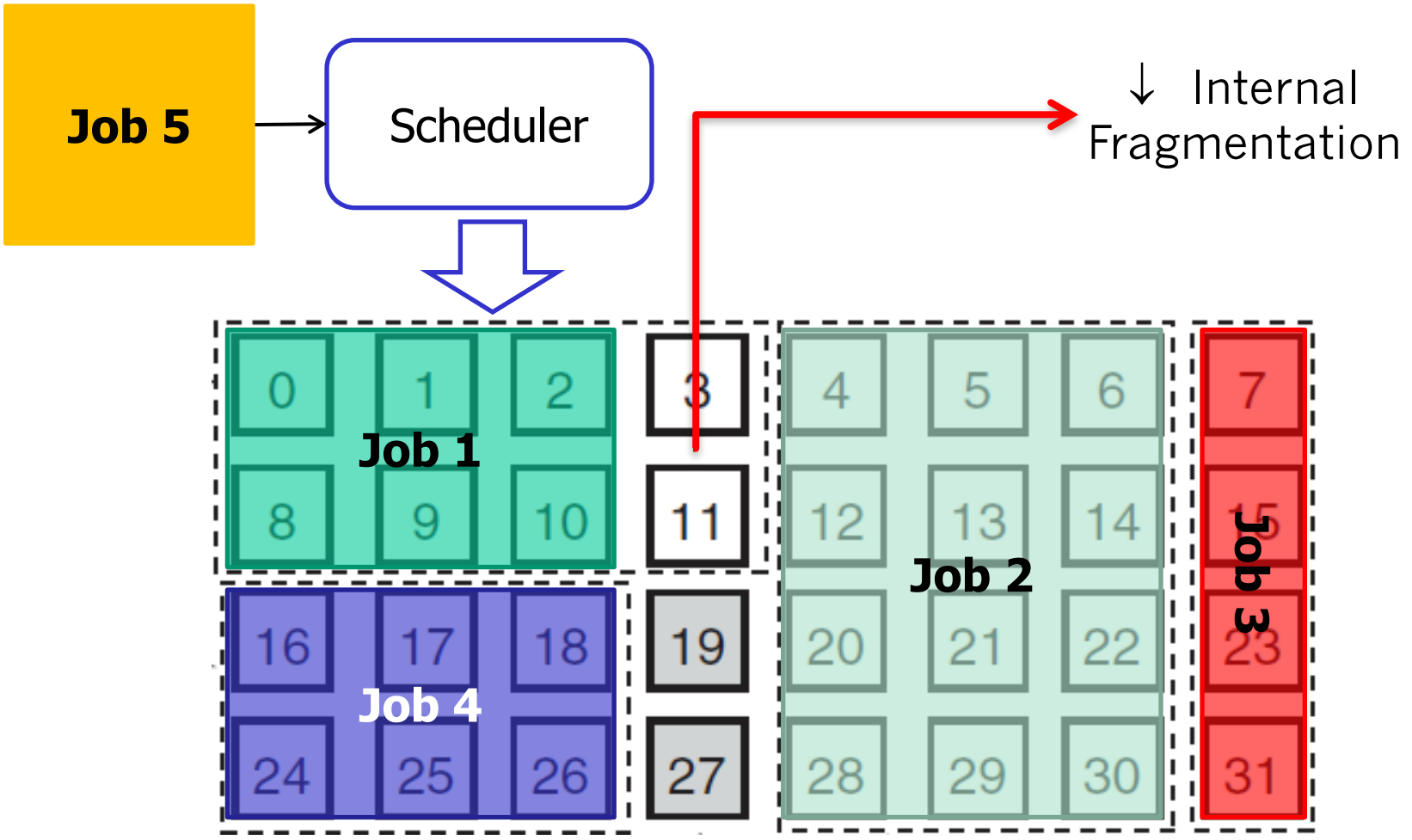
↑ Simple to implement

↑ Preserves data locality

↓ Arbitrary queueing

↓ Internal fragmentation (unused PEs within a partition)

Space-sharing



Space-sharing

b) Variable partitioning

- Partition size is set at submission time according to incoming job request
- Partition size does not change during job runtime

↑ Matches job request

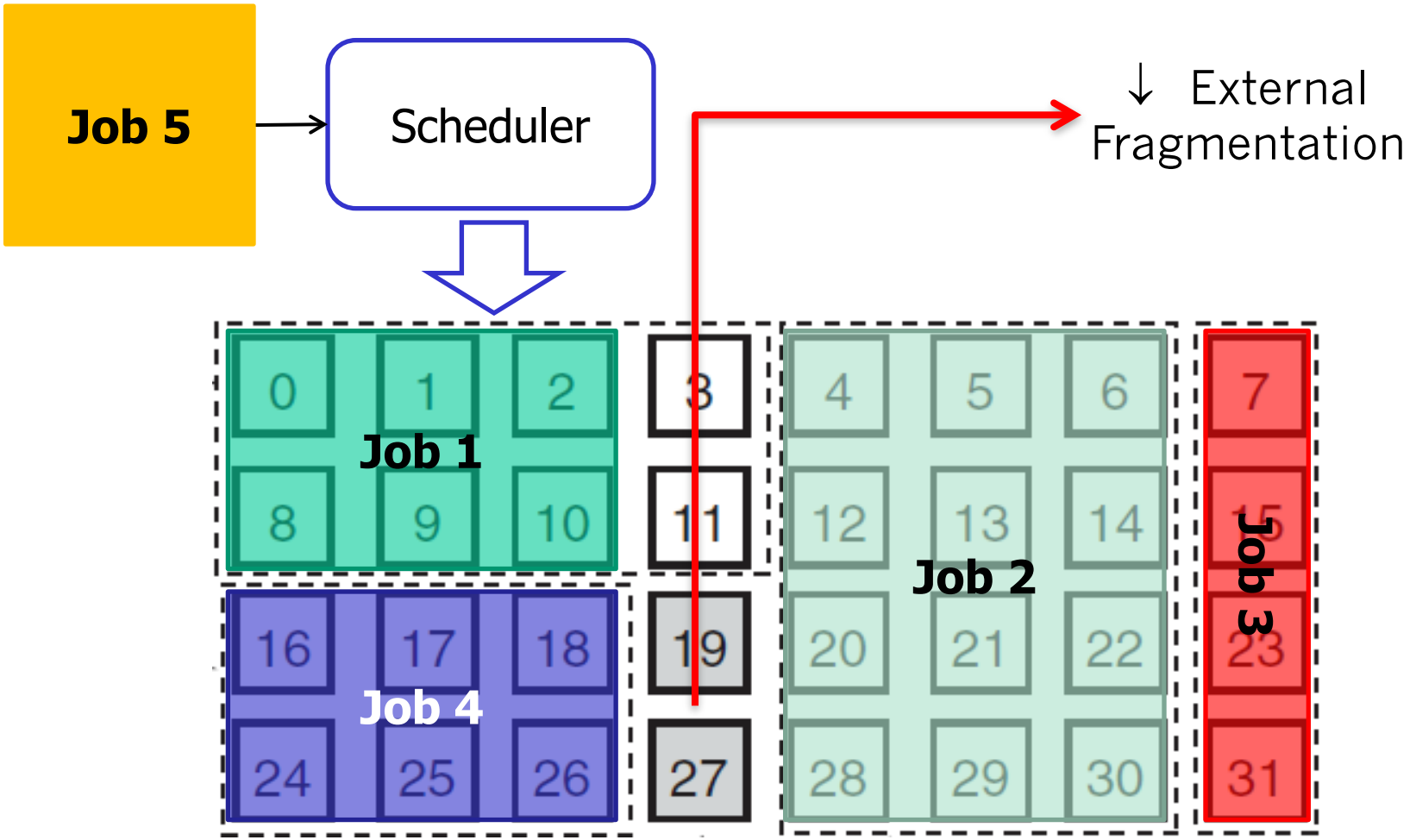
↑ Preserves data locality

↓ Arbitrary queueing

↓ External fragmentation (PEs not allocated to any partition)

↓ Also internal fragmentation if the architecture imposes restrictions to the partition size

Space-sharing

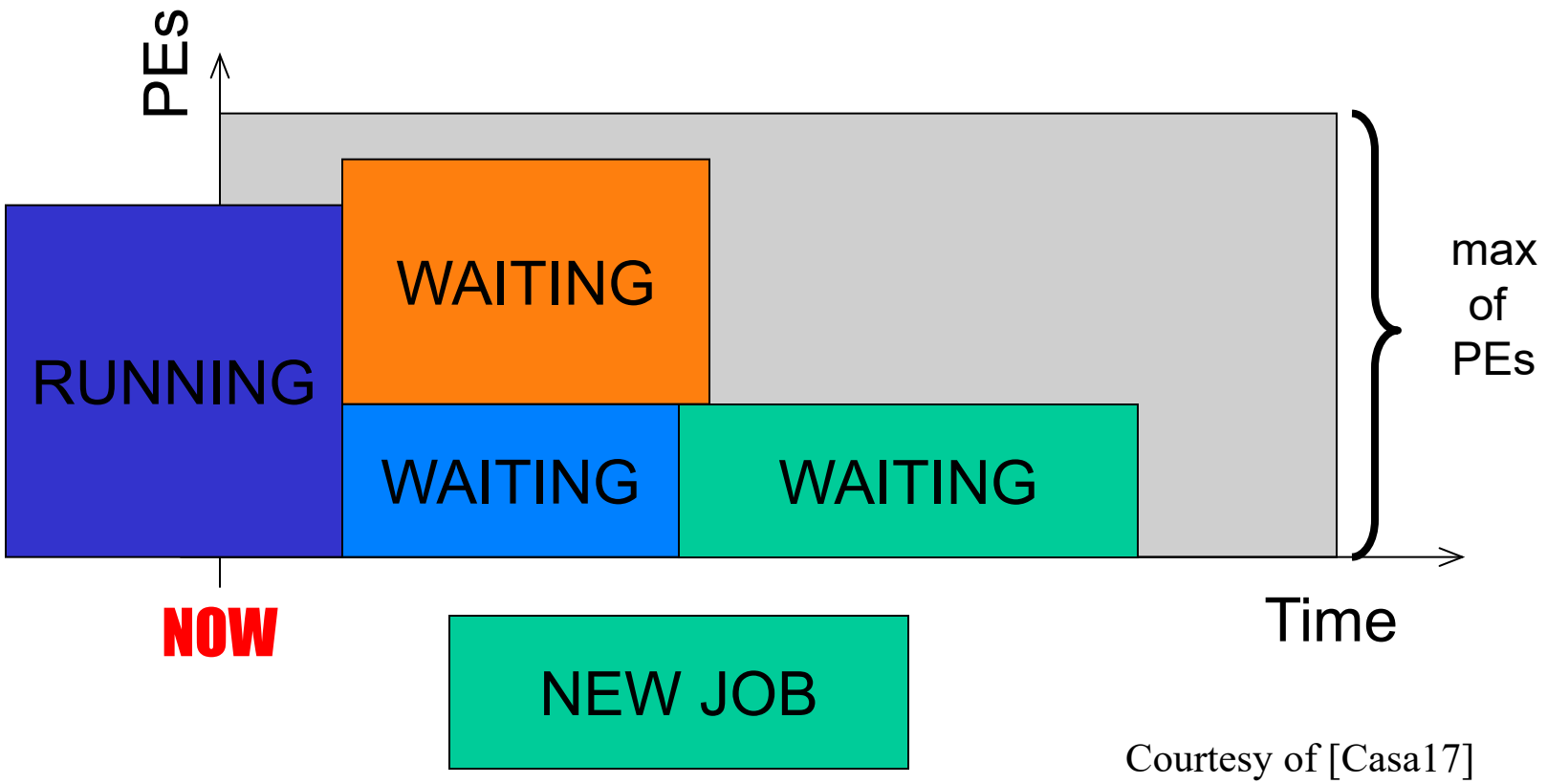


Scheduling policies

- Jobs are queued until sufficient PEs become available for them to run
  - Alternatively, combine with time-sharing to schedule multiple jobs within a partition
- Which process should be run next?
  - Algorithms for scheduling queued jobs onto partitions are needed
    - FCFS (First Come First Served)
    - SJF (Shortest Job First)
    - Backfilling-based policies
    - etc.

Scheduling policies

- Graphical representation of a schedule



Courtesy of [Casa17]

Scheduling policies: FCFS

- Activated at job arrival and job termination
  - **If** enough free PEs to start FIRST job in queue
  - **Then** start job
  - **Else** wait

↑ Very simple to implement

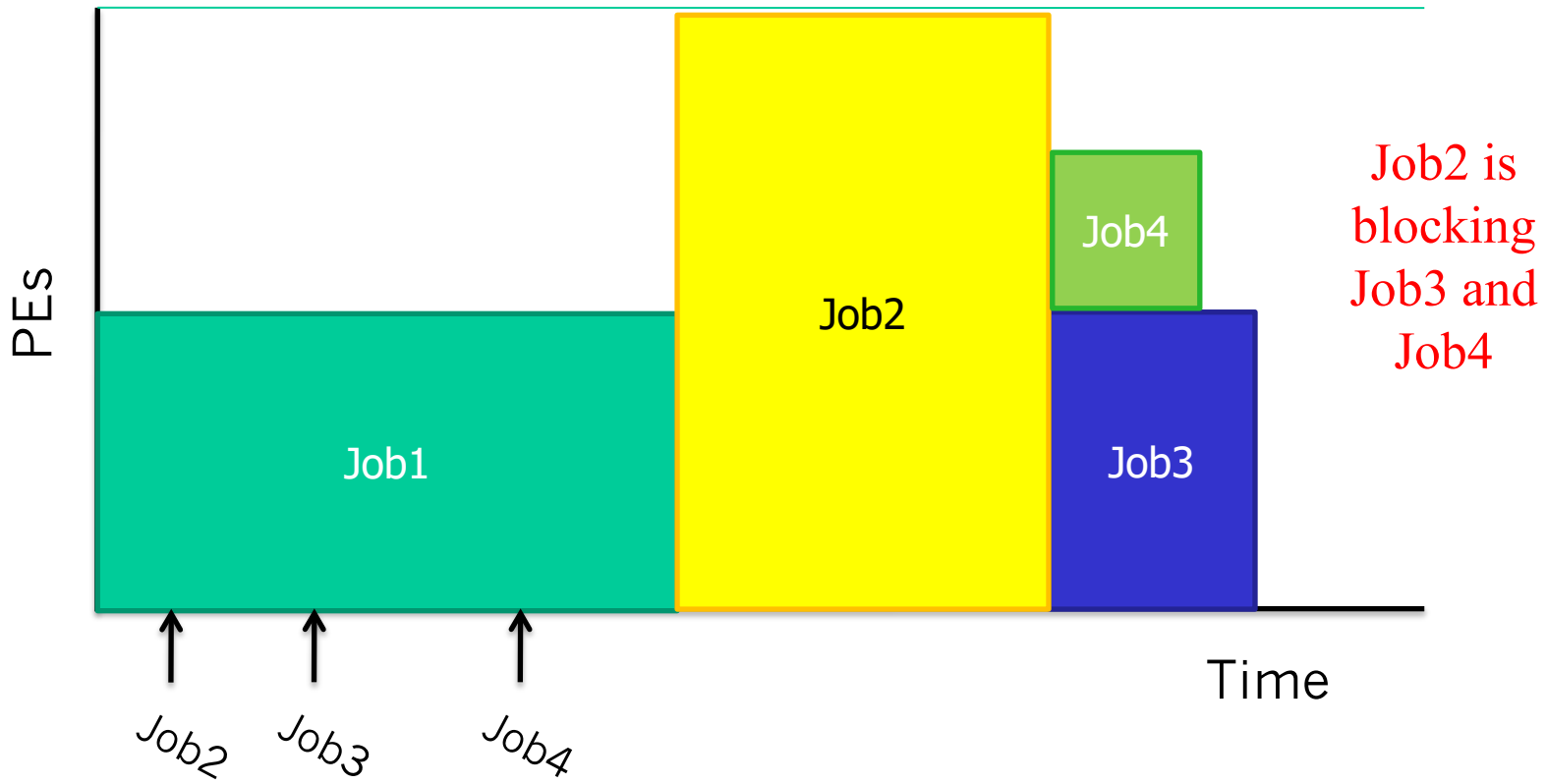
↑ Fairness

↓ External fragmentation

  - The first job blocks the rest of the queue

Scheduling policies: FCFS

- Example

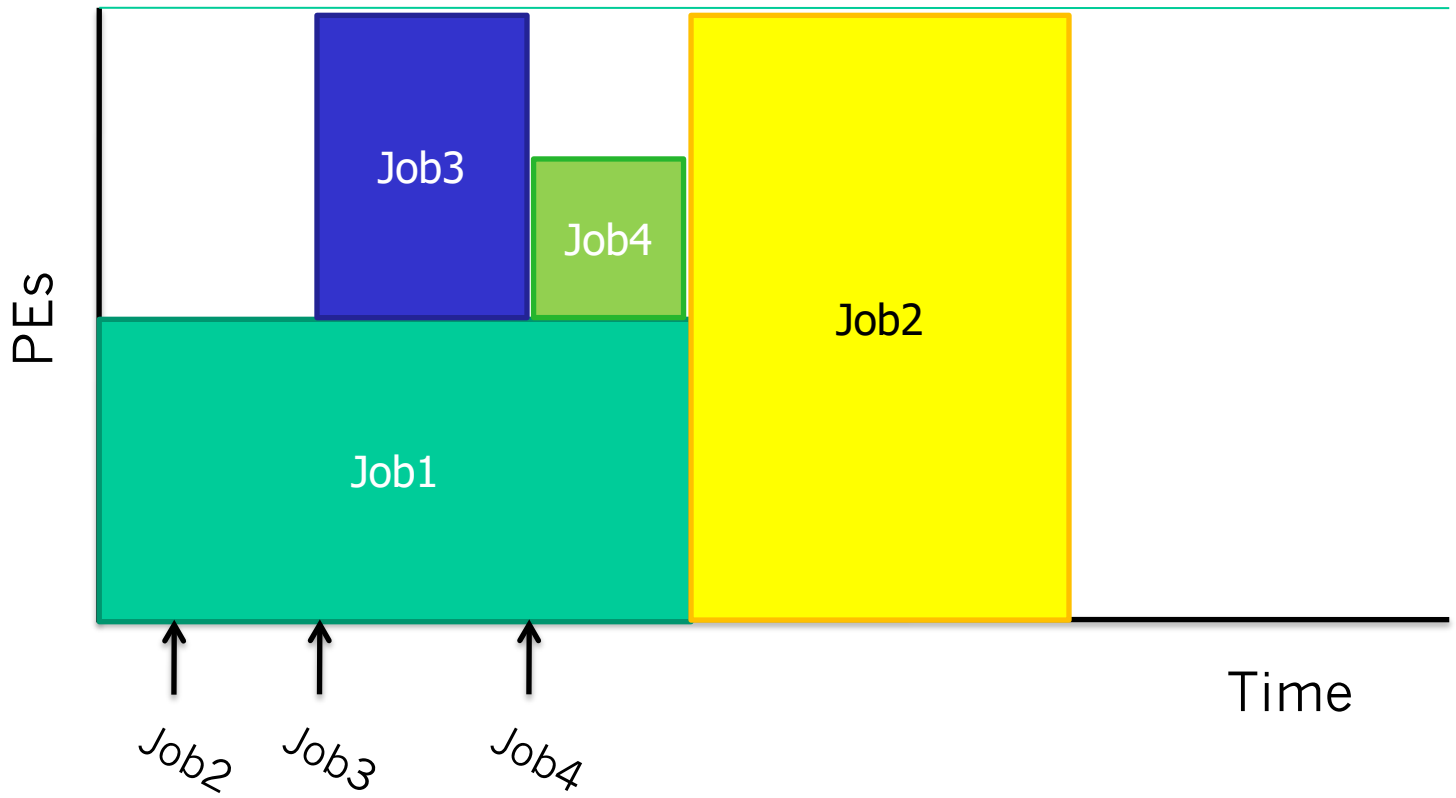


Scheduling policies: backfilling

- IDEA: Backfill jobs that arrived later to the head of the queue ('fill holes')
  - Reduce fragmentation by breaking FCFS order
- Restriction: Defines N reserved jobs and backfilled jobs cannot delay reserved ones
- Requires job runtime estimations and number of required PEs
- Backfilling can happen whenever a job arrives or terminates

Scheduling policies: backfilling

- Example

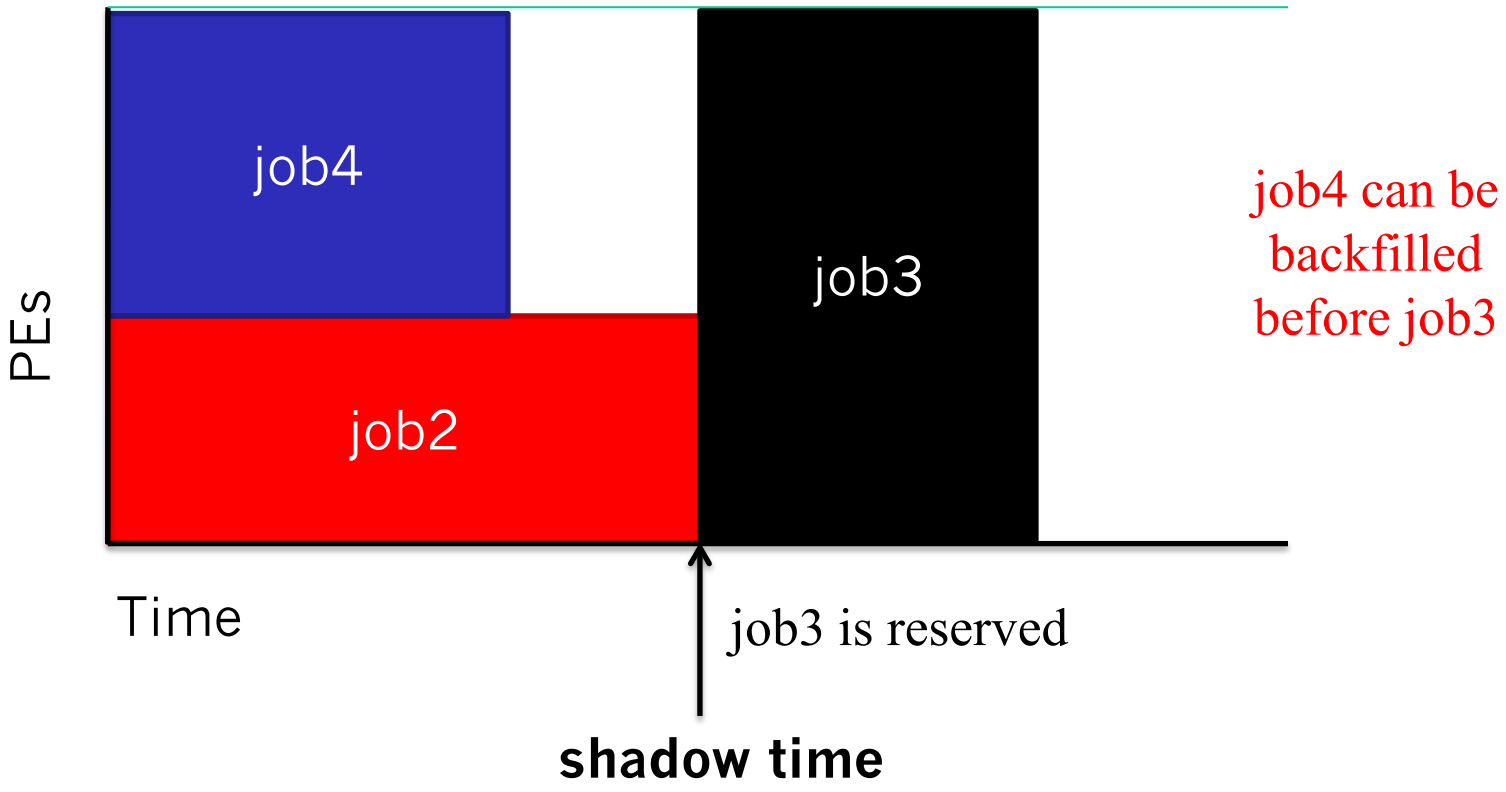


Scheduling policies: EASY backfilling

- Queue is ordered in FCFS order
- Only the FIRST queued job gets a reservation
  - This guarantees that it is not delayed
- Go through the queue in order starting with the second job, and backfill a job if:
  a) It will terminate by the shadow time, **OR**
     - 'Shadow time' refers to the expected start time of the first queued job
  b) It needs at most extra PEs
     - 'Extra PEs' refers to the number of available PEs when the first queued job starts running

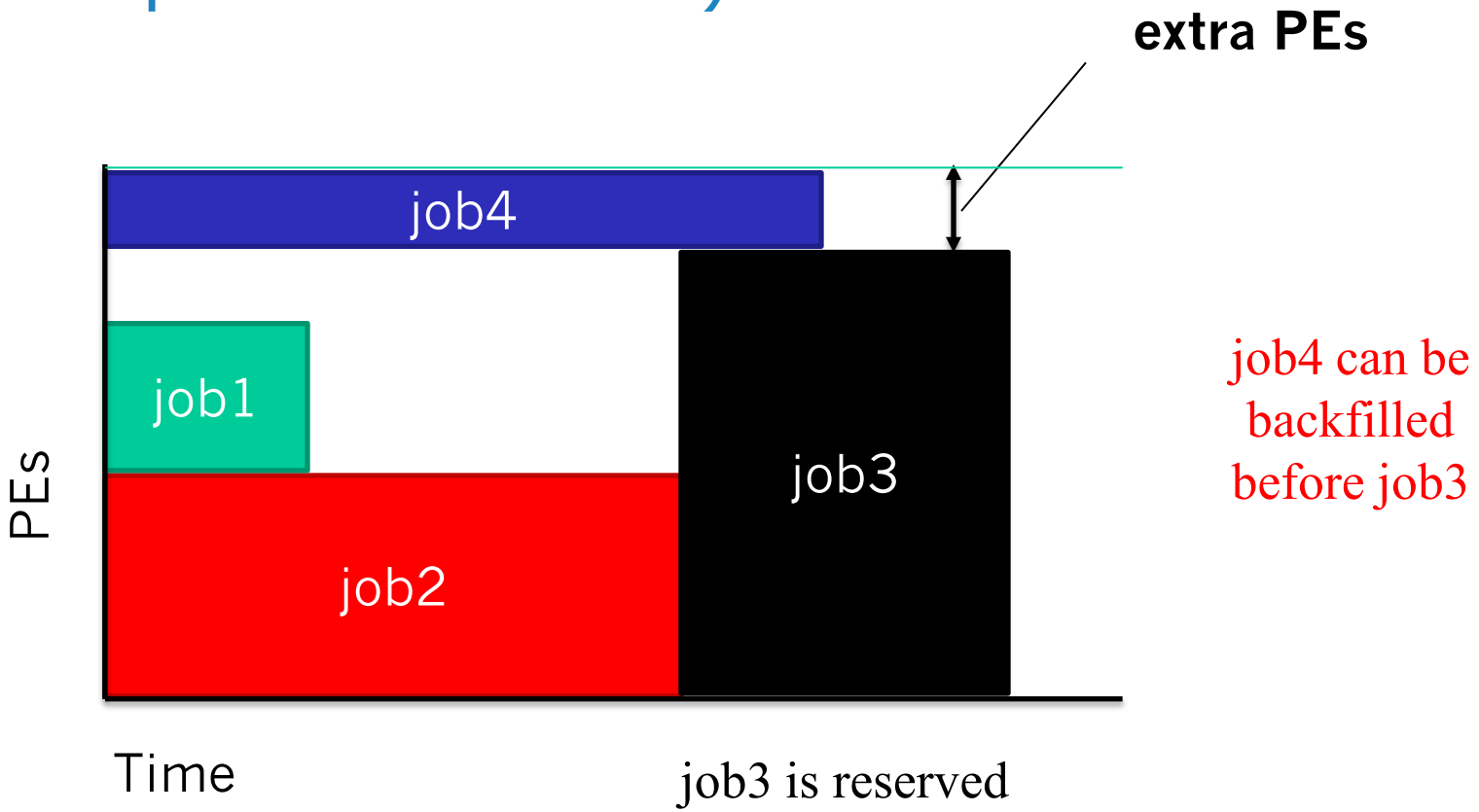
Scheduling policies: EASY backfilling

- Example of condition a)



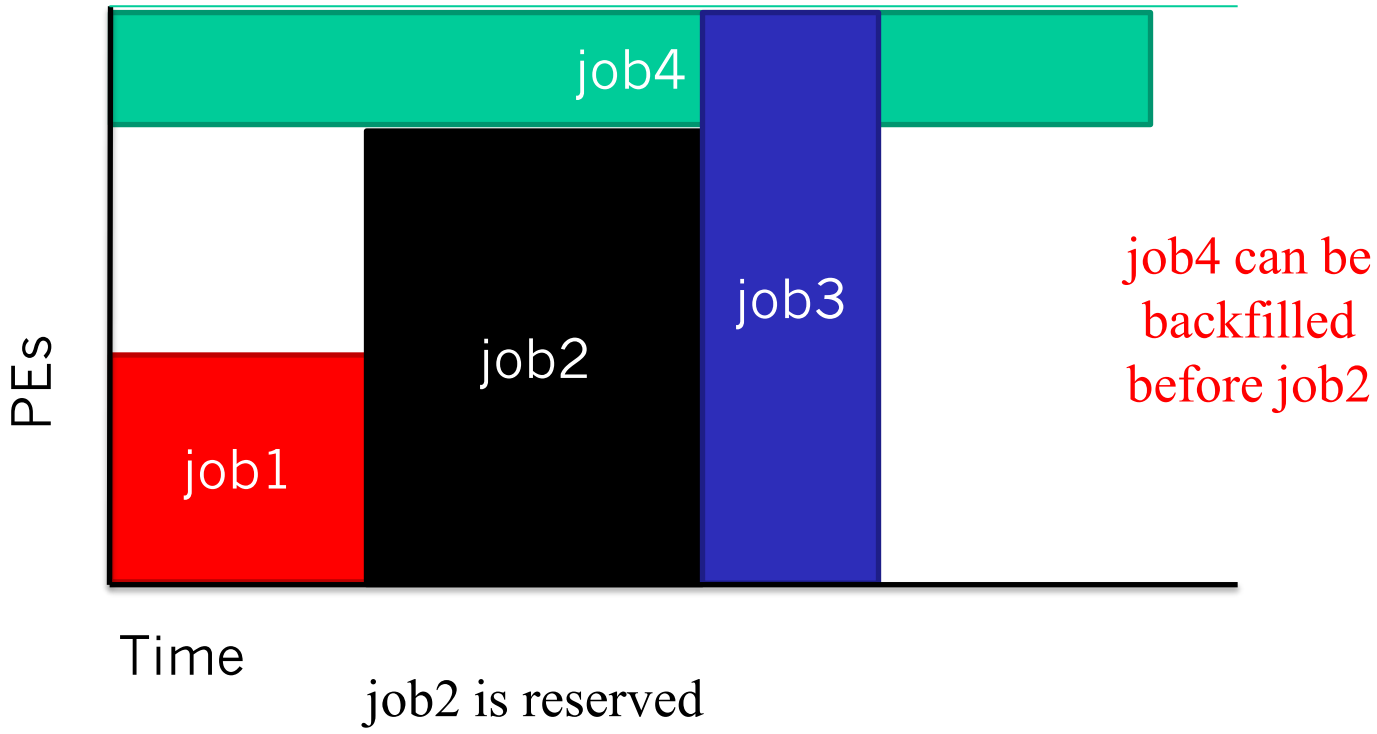
Scheduling policies: EASY backfilling

- Example of condition b)



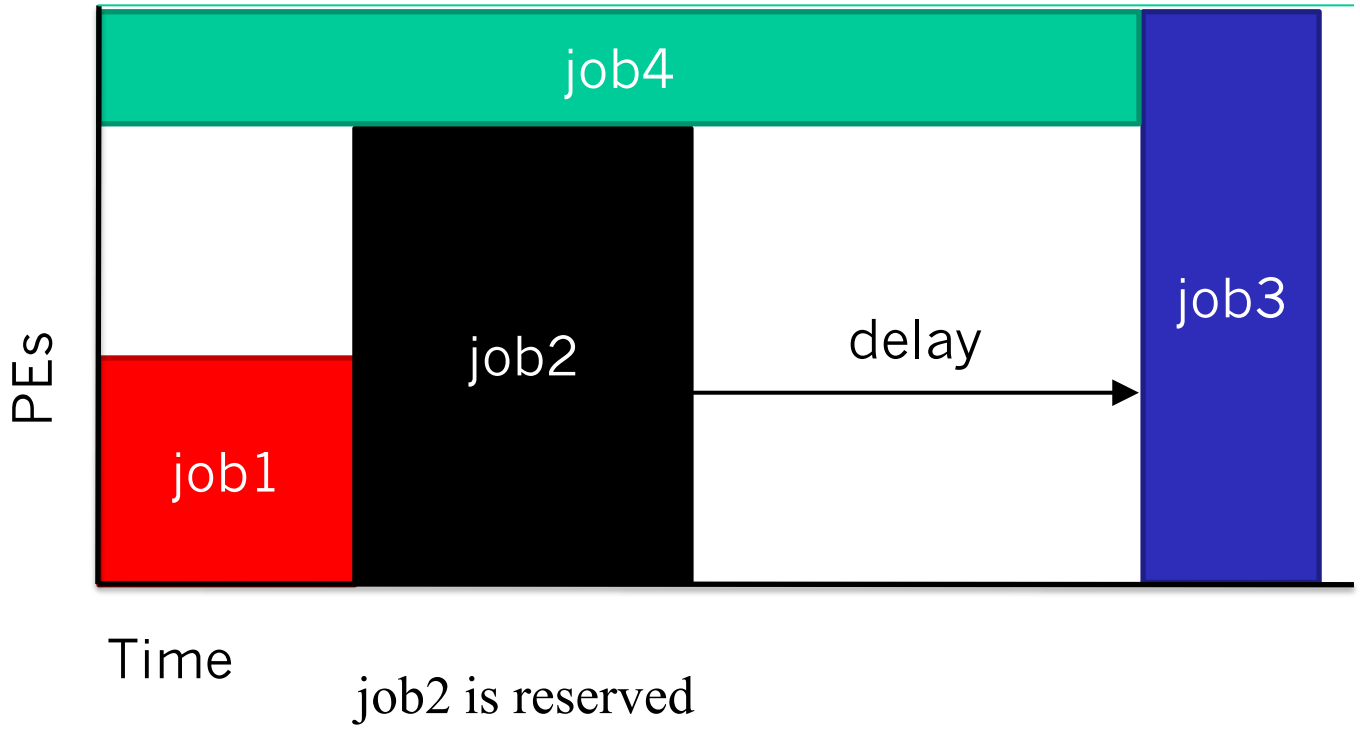
Scheduling policies: EASY backfilling

- The first queued job (e.g. job2) will never be delayed by backfilled jobs …



Scheduling policies: EASY backfilling

- … BUT other queued jobs may be delayed (e.g. job3)



Policies: backfilling variations

- Order of queued jobs
  - FCFS (EASY backfilling), priorities (slack-based backfilling), shortest estimated runtime first (SJF backfilling)
- Number of reserved jobs
  - 1 (EASY backfilling), all (conservative backfilling), adaptive (selective backfilling: only reserve jobs whose expected slowdown exceeds a threshold)
- Look-ahead into the queue (i.e. how many jobs are analyzed simultaneously to backfill?)
  - One job at a time, the whole queue at once

Scheduling policies: backfilling

- All backfilling decisions are based on job runtime estimates provided by users
  - Typically, a job is killed if it goes over (to avoid delaying other jobs)
- Trade-off:

A. Provide an aggressive estimate
  - Job can go through the queue faster (may be backfilled more easily), but it may be killed

B. Provide a conservative estimate
  - Job is harder to backfill, but it will not be killed

Space-sharing

c) Adaptive partitioning

- Partition size is set when the job starts based on system load (taking job request into account) and it does not change during job execution

↑ Preserves data locality

↑ Adapts to load

↑ Improved efficiency

↓ Requires moldable jobs: number of processes is determined at start time

↓ Wasted PEs if job goes through several phases

↓ Queueing if not free PEs when new job arrives

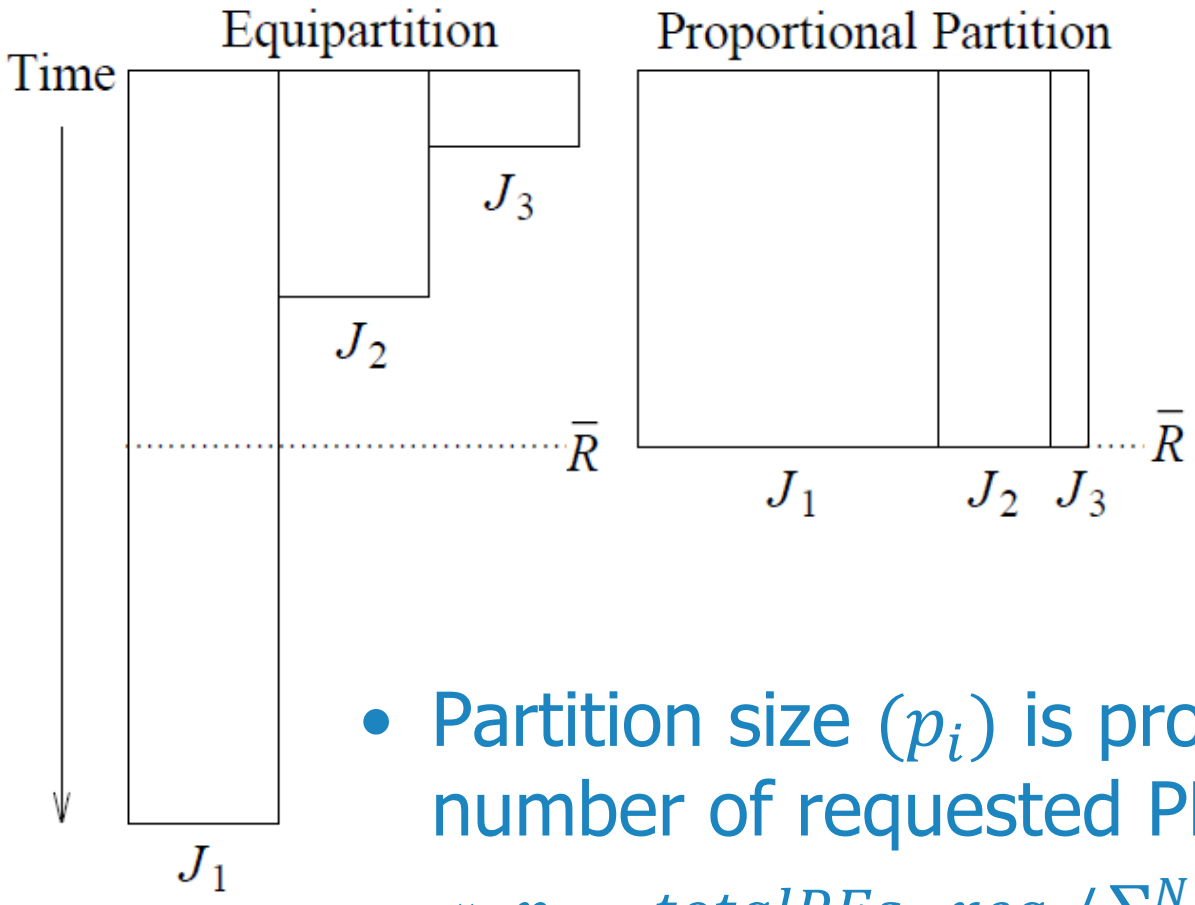
Scheduling policies: equipartition

- Aims for equal-sized partitions for all jobs
- Applied at job arrival/termination
  1. Initial allocation to all waiting jobs is set to zero
  2. Increase the PEs allocated to each job by one
     - Any job whose allocation reaches its request drops out
  3. Go on while there are free PEs and waiting jobs

  » A new job is allocated the lesser of the number of requested PEs and the number of free PEs
     - Queue the job if there are not free PEs

  » PEs of a finished job are equally divided among the jobs in the queue

Policies: proportional partitioning



policy	job	req_i	p_i	rt_i
EQUI	J_1	1000	20	50
	J_2	400	20	20
	J_3	100	20	5
PROP	J_1	1000	40	25
	J_2	400	16	25
	J_3	100	4	25

number of PEs = 60

- Partition size (p_i) is proportional to the number of requested PEs (req_i)
  - » $p_i = totalPEs \cdot req_i / \sum_1^N req_j$

Space-sharing

d) Dynamic partitioning

- Partition size can change during job execution to respond to load changes (new/terminated jobs and changing requirements of a running job)

↑ No fragmentation

↑ Adapts to load

↑ Improved efficiency

↓ Requires malleable jobs: processes change at runtime according to the availability of PEs

↓ Overhead of re-partitioning

↓ Does not preserve data locality

Scheduling policies: equipartition

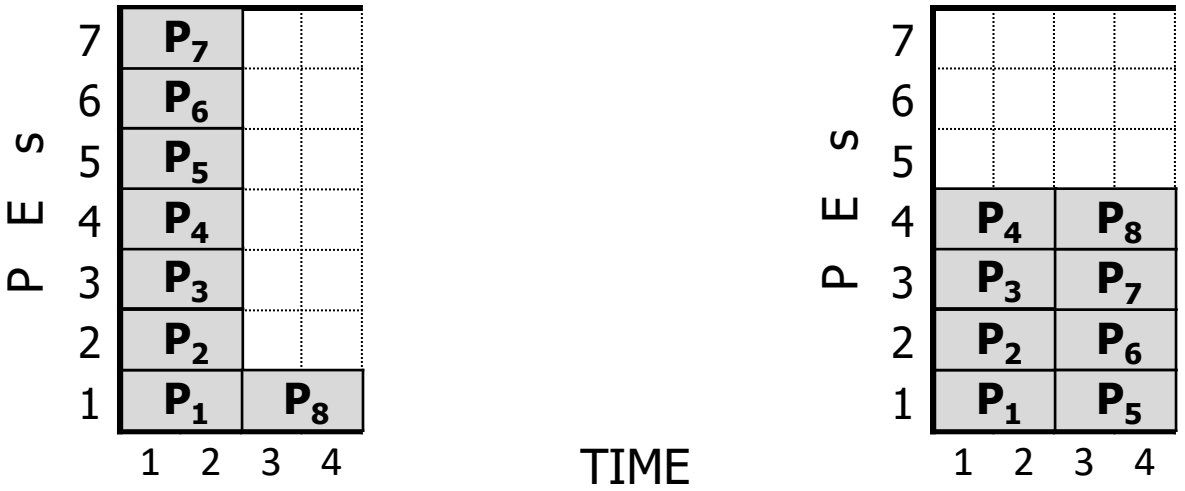
- Aims for equal-sized partitions for all jobs
- Applied at job arrival/termination and when job requirement changes
  1. Initial allocation to **all jobs** is set to zero
  2. Increase the PEs allocated to each job by one
     - Any job whose allocation reaches its request drops out
  3. Go on while there are free PEs and jobs

  » Some PEs might be preempted from running jobs and given to other jobs

  » PEs of a finished job are equally divided among ALL the other jobs

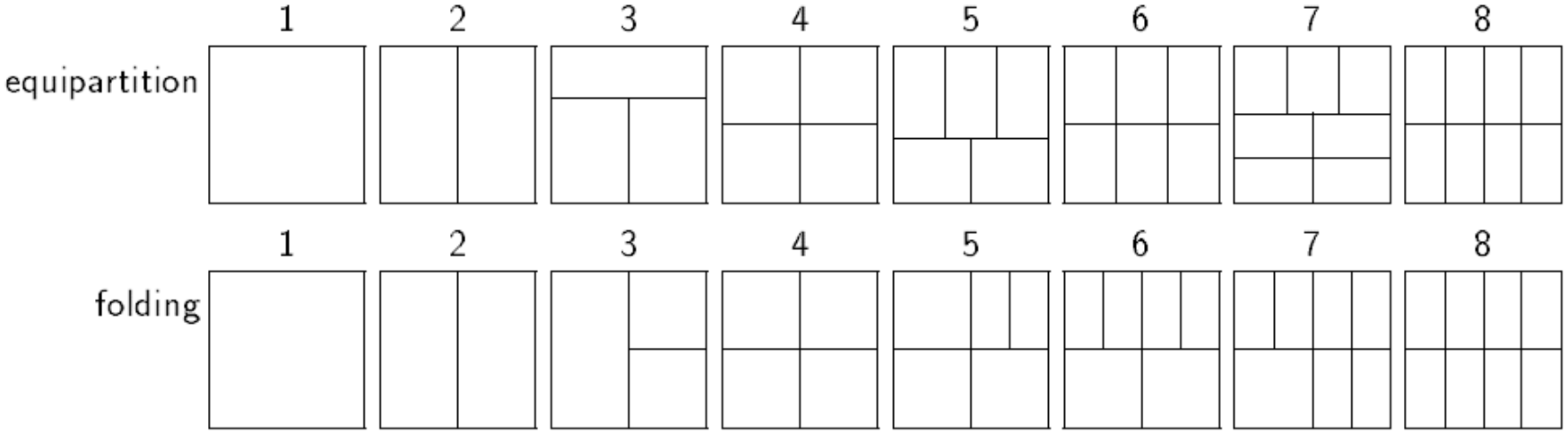
Policies: efficiency preservation

- Consider runtime parameters such as the achieved speed-up or the efficiency of the jobs to decide the partition size
  - e.g. running a job with 8 processes on 7 PEs does not provide any benefits over using only 4 PEs



Policies: efficiency preservation

- **Folding** policy for efficiency preservation
  - On job arrival, the largest current partition is folded in half, freeing PEs for the new job
    - Assumes that total number of PEs is a power of two, and jobs are designed to execute optimally with all the PEs
  - Processes on each partition will time-share the PEs



Courtesy of [Feit97]

Contents

- Introduction
- **Process scheduling**
  - Introduction
  - Scheduler architectures
  - Time-sharing
  - Space-sharing
  - **PE allocation**
  - Gang scheduling
  - Metrics
- Process resilience
- Virtualization

PE allocation

- Selection of the specific subset of PEs to use

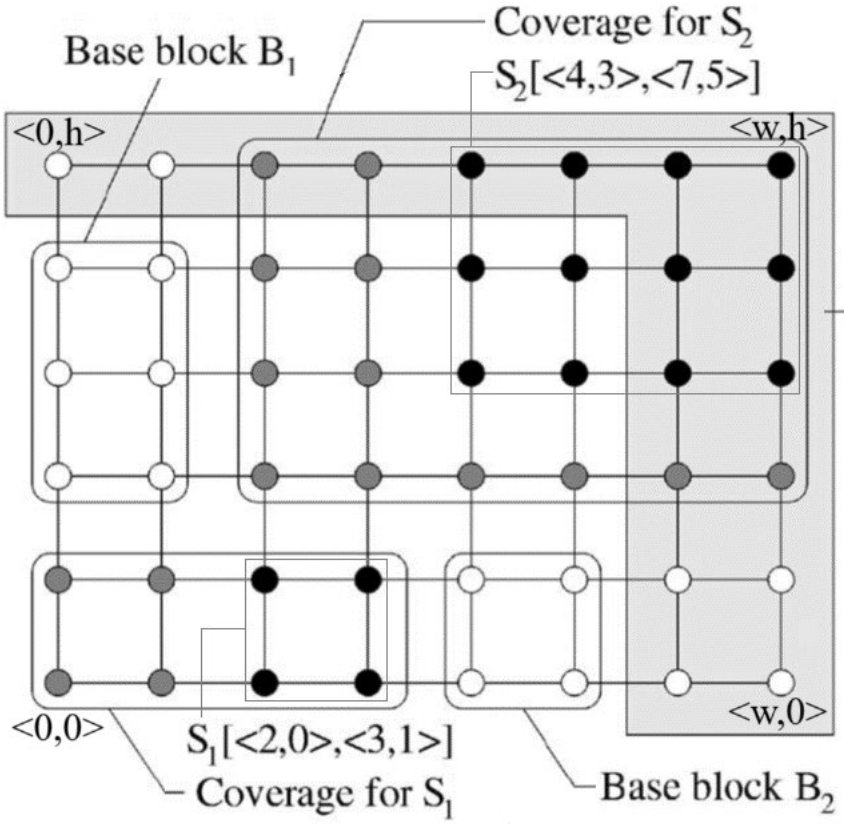
A. Contiguous partitions
  - PEs are constrained to remain physically adjacent
    - To minimize the communication cost inside the partition

  a) PEs must form a subgraph of the original architecture (e.g., rectangles in a 2D mesh)
    - Suffers from internal and external fragmentation

  b) Form-free partitions of arbitrary shapes (e.g., Leak, based on the principle of leaking water)
    - Avoids internal fragmentation and reduces external one, but the latter could be still significant

PE allocation: Rectangular

Base blocks B_i for job J(3,2)
PEs that can be used as base of a free submesh to run job J



(Busy) submeshes S_i
Submesh S_i[base, end] is identified by its lower-left (base) and top-right (end) PEs

Reject area for job J(3,2)
The use of any of those PEs as the base will make job J cross the boundary of the mesh

Coverages for submeshes S_i for job J(3,2)
The use of any of those PEs as the base will make job J overlap with Si

PE allocation: Rectangular

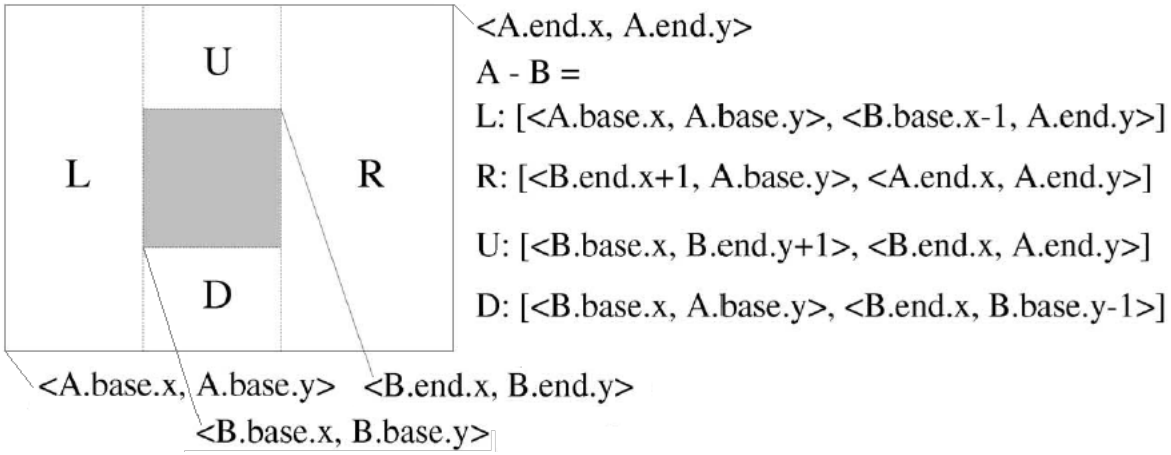
- Find a free submesh (base block) to run job J(p,q)

1. Determine the reject area and the coverages of all the busy submeshes for job J(p,q) through coordinate calculation

⇒ Reject area: $\mathrm{PE}(x, y)$ such that $(w - p + 1 \leq x < w)$ or $(h - q + 1 \leq y < h)$

⇒ Coverage S[<x, y><x', y'>]: [<max(0, $x - p + 1$), max(0, $y - q + 1$)><x', y'>]

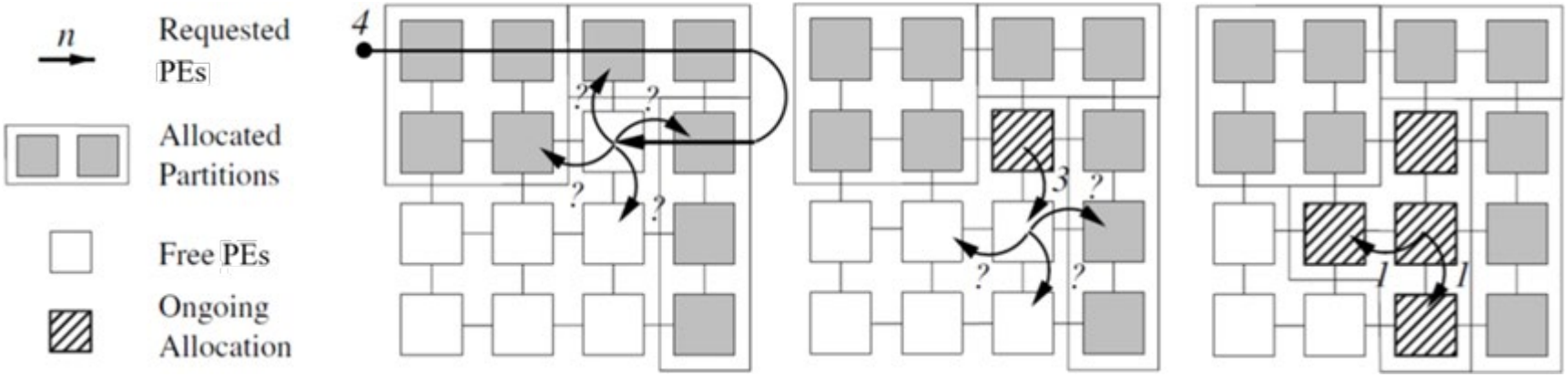
2. Obtain the base blocks by spatially subtracting the reject area and all the coverages from the set of all the PEs



3. Choose one of the base blocks (using first-fit, best-fit, ...)

PE allocation: Form-free: Leak

- Search for a free PE to act as origin using a 'snake' wave
- Test all direct neighbors of the origin PE to check if any are free ⇒ Allocate each free neighbor to the partition
- Continue recursively until all the needed PEs are allocated
- If no suitable partition can be formed from this origin PE, try next free PE in the search wave as origin
- Allocation fails if no more PEs are available to try



Courtesy of [RoHL07]

PE allocation

B. Non-contiguous partitions

↑ External fragmentation reduced considerably

- Job can be always allocated if there are enough free PEs

↑ Better compatibility with dynamic partitioning

↓ Overhead due to increased message latency and network contention, which will depend on the degree of non-contiguity (i.e., PE dispersal)

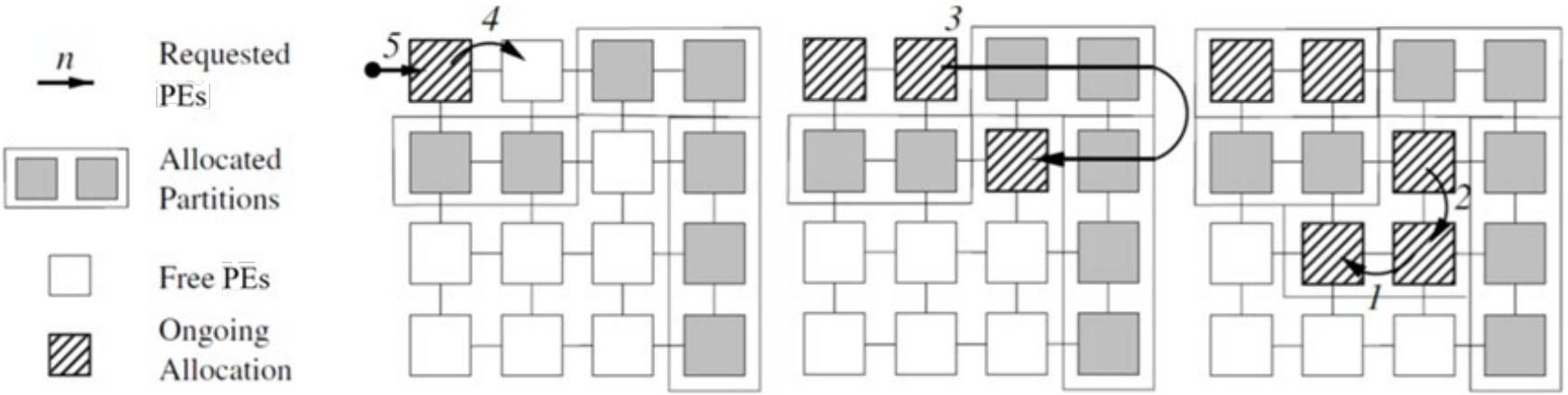
a) Randomly allocated PEs

b) Combination of several contiguous partitions

- e.g., Leak

PE allocation: Non-contiguous: Leak

- Same as the contiguous version, except that a partial allocation insufficient to fulfill the request is kept allocated while the search wave continues from the next origin PE looking for the additional PEs needed

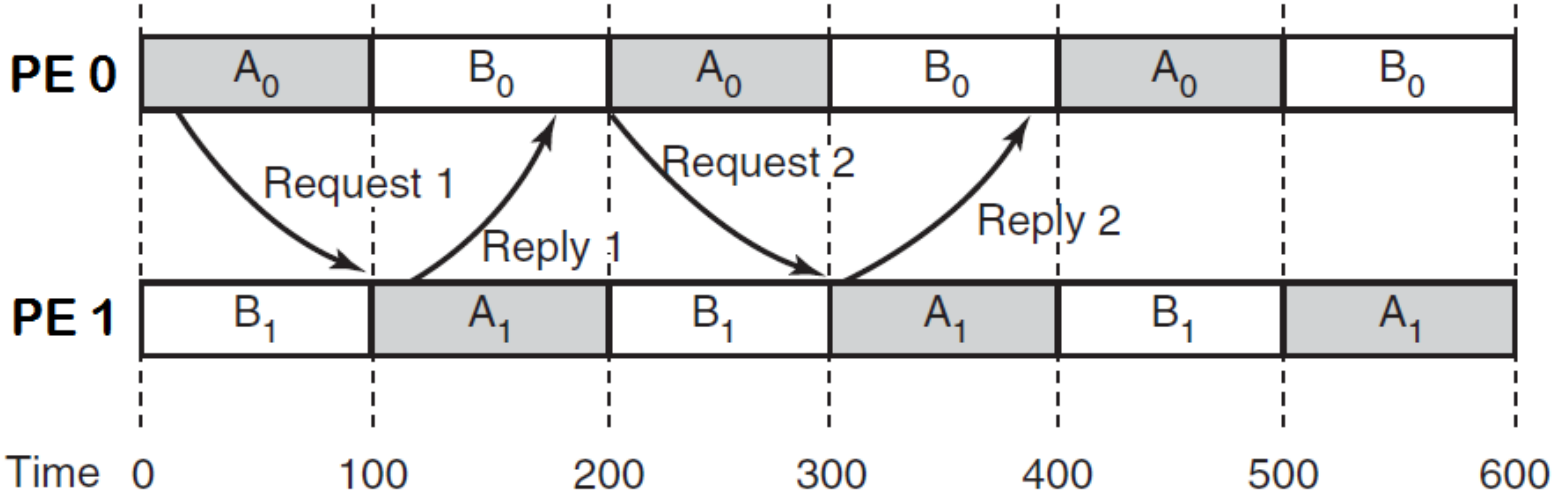


Contents

- Introduction
- **Process scheduling**
  - Introduction
  - Scheduler architectures
  - Time-sharing
  - Space-sharing
  - PE allocation
  - **Gang scheduling**
  - Metrics
- Process resilience
- Virtualization

Motivation

- When using time-sharing, the performance of parallel jobs could be degraded, as processes could be preempted during the interactions



- Aim for policies that schedule all processes of a parallel job together ⇒ **Gang scheduling**

Gang scheduling

- Processes in a job are grouped into a **gang**
- Processes in a gang execute simultaneously on distinct PEs, using a one-to-one mapping
- Time-sharing is used, with all the processes in a gang being preempted and rescheduled at the same time (as a unit)
  - All PEs are scheduled synchronously. This means that time is divided into discrete quanta
  - At the start of each new quantum, all the PEs are rescheduled, with other process being started on each one

Gang scheduling: Ousterhout matrix

- Mapping of gangs to PEs over time
  - Each gang is entirely packed into a single row

PEs

quanta (time slots)

	P0	P1	P2	P3	P4	P5
Q0	A0	A1	A2	A3	A4	A5
Q1	B0	B1	B2	C0	C1	C2
Q2	D0	D1	D2	D3		
Q3	E0	E1	E2	E3	E4	
Q4	A0	A1	A2	A3	A4	A5
Q5	B0	B1	B2	C0	C1	C2
Q6	D0	D1	D2	D3		
Q7	E0	E1	E2	E3	E4	

→ external

→ fragmentation

Gang scheduling

- Advantages
  - ↑ Similar environment to a dedicated machine, in which all processes of a job progress together
  - ↑ Improves utilization
- Disadvantages
  - ↓ Overhead for context switch and coordination
  - ↓ May have some idle PEs (external fragmentation)
    - Based on the PE allocation (packing) algorithm, namely:
    - How are new jobs mapped to time slots?
    - Which PEs are used within the chosen slot?
    - Is job/slot/PE mapping reconsidered when jobs finish?

Gang scheduling

- First Fit and Best Fit packing algorithms can be used to map jobs (1-by-1) to time slots

A. First Fit
   - Used slots are scanned in sequential order
   - The first one with sufficient capacity is chosen
   - If none has enough capacity, a new one is opened

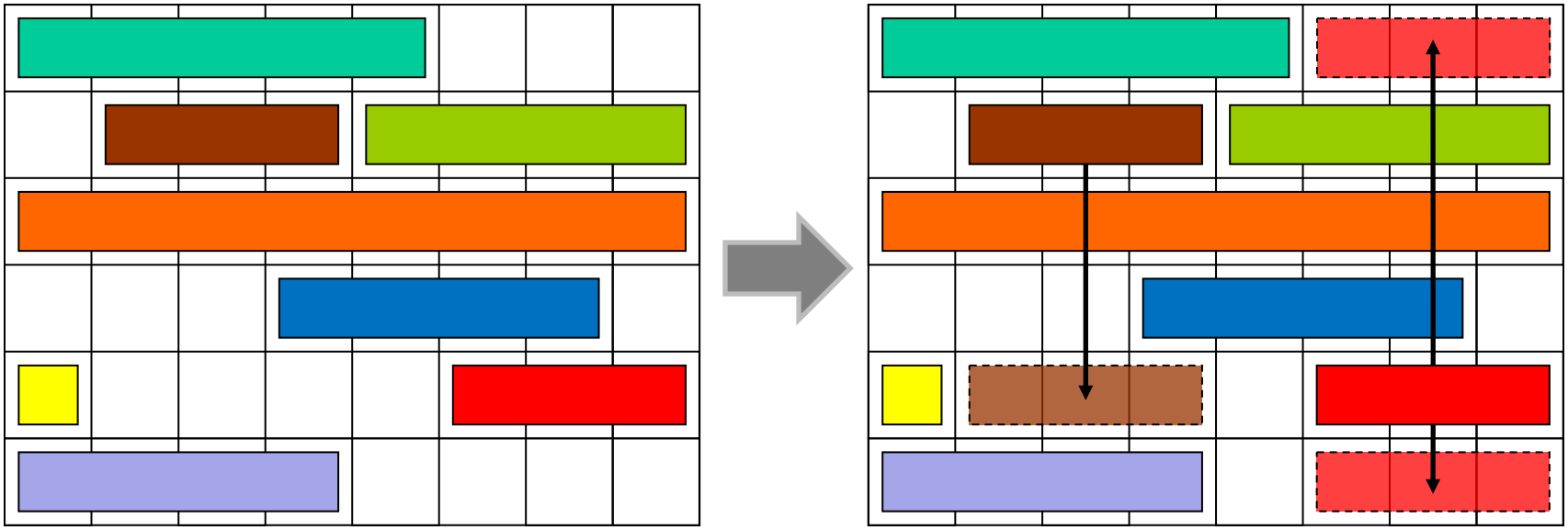
B. Best Fit
   - Used slots are sorted according to their free capacity
   - The one with the smallest capacity that is sufficient is chosen
   - If none has enough capacity, a new one is opened

Gang scheduling

- A job is usually assigned to contiguous PEs to facilitate slot unification when jobs terminate
- A job is usually assigned to the same PEs to exploit data locality (affinity scheduling)
- Left-Right packing variants (start using PEs from left/right) to lower overlap between jobs
- The goal is to promote alternative scheduling and slot unification to reduce fragmentation

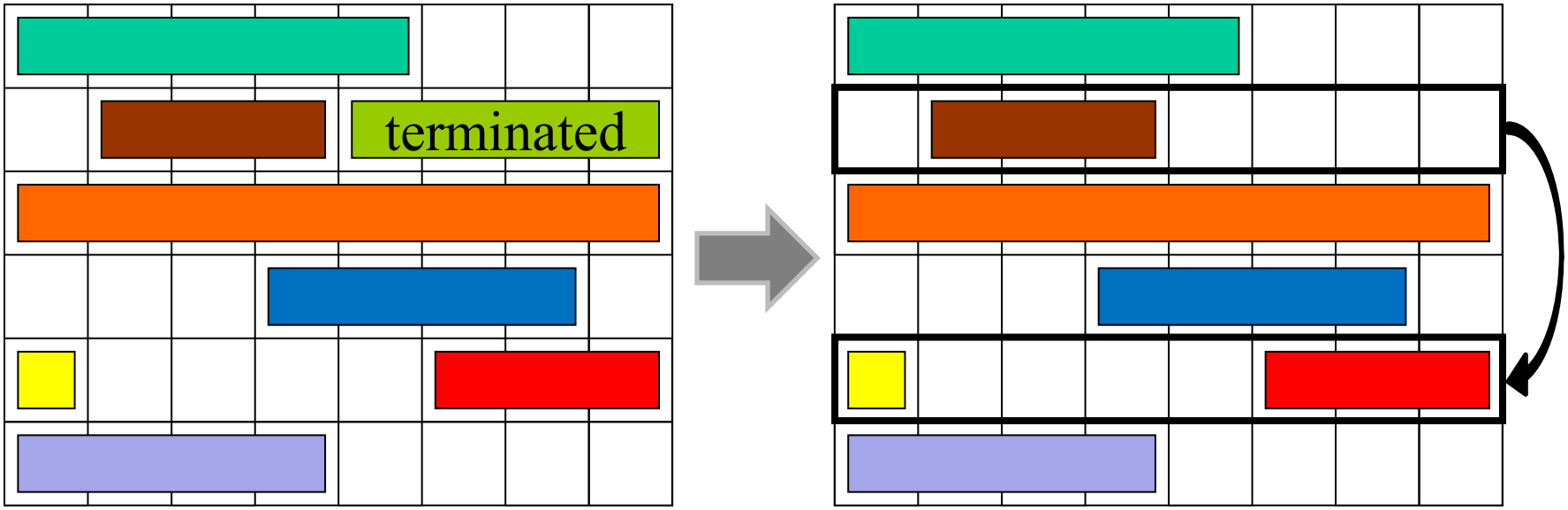
Gang scheduling

- Alternative scheduling: schedule a job in multiple slots if corresponding PEs are free
  - ↑ Reduces fragmentation
  - ↓ Unfair (increases run fraction of this job only)



Gang scheduling

- Slot unification: merge slots in which jobs use complementary PEs
  - ↑ Reduces fragmentation
  - ↑ Increases run fraction of ALL the jobs



Contents

- Introduction
- **Process scheduling**
  - Introduction
  - Scheduler architectures
  - Time-sharing
  - Space-sharing
  - PE allocation
  - Gang scheduling
  - **Metrics**
- Process resilience
- Virtualization

Scheduling policies: metrics

- Metrics to quantify how good a schedule is
- It has to be an aggregate metric over all jobs

a) **Wait time**: time spent in the queue waiting to be scheduled = T_W
   - Job1 waits 1h to run 100h
   - Job2 waits 1h to run 1s

   ⇒ Both jobs have the same waiting time (1h), but Job1 is really happy and Job2 is not happy at all

   ➢ Does not capture users' notion of responsiveness (i.e., users expect their jobs to take time that is proportional to their length)

Scheduling policies: metrics

b) Response time (turnaround time or flow): time elapsed from job submission until its completion (waiting + running) = $T_W + T_R$

- Job1 waits 1s to run 1h
- Job2 waits 1h to run 1s

⇒ Both jobs have the same response time (1h1s), but Job1 is really happy and Job2 is not happy at all

- ➢ Responsiveness is still not adequately captured
- ➢ Calculation of the mean places greater emphasis on long jobs and is not much affected if a short job waits a few minutes

Scheduling policies: metrics

c) **Slowdown** (or stretch): $(T_W + T_R) / T_R$

- Job1 waits 1s to run 1h; slowdown = 3601/3600
- Job2 waits 1h to run 1s; slowdown = 3601/1

⇒ Job1 is really happy (slowdown is low) and Job2 is not happy at all (slowdown is high)

- Quantifies loss of performance due to competition for PEs and captures better the responsiveness
- Calculation of the mean overemphasizes the importance of very short jobs ⇒ see Example 1
- Almost equivalent jobs (doing the same amount of work with the same response time) may have very different slowdown ⇒ see Example 2

Scheduling policies: metrics

- Example 1
  - Job1 waits 10min to run 1s; slowdown = 601/1
  - Job2 waits 10min to run 100s; slowdown = 700/100
  - ⇒ Both jobs have the same waiting time (10min), but the slowdown of the short job (Job1) is considerably higher
  - ➢ Bounded-slowdown to reduce the influence of short jobs
- Example 2
  - Job1 waits 0s to run 100s on 1 PE; slowdown = 100/100
  - Job2 waits 90s to run 10s on 10 PEs; slowdown = 100/10
  - ⇒ Both jobs have the same response time (100s) and CPU time (100s x 1PE = 10s x 10PE), but their slowdown differs
  - ➢ Per-processor-slowdown to normalize the slowdown for jobs that perform the same amount of work with different PEs

Scheduling policies: metrics

d) Bounded-slowdown: for short jobs, this measures the slowdown relative to some threshold τ rather than to the actual runtime = $\max\{(T_W + T_R) / \max\{T_R, \tau\}, 1\}$

- τ: interactivity threshold (the time users are willing to wait)
- Job1 waits 10min to run 1s; b-slowdown (τ=10) = 601/10
- Job2 waits 10min to run 100s; b-slowdown = 700/100

e) Per-processor-slowdown: divides the slowdown by the number of PEs used

- Job1 waits 0s to run 100s on 1 PE; pp-slowdown = 1
- Job2 waits 90s to run 10s on 10 PEs; pp-slowdown = 1

Contents

- Introduction
- Process scheduling
- **Process resilience**
- Virtualization

Contents

- Introduction
- Process scheduling
- **Process resilience**
  - **Introduction**
  - Redundancy, state machines, and quorums
  - Recovery strategies
- Virtualization

Introduction

- Failures on a desktop system often affect the **entire** system
- Processes on a WSC can fail independently, thus failures are typically **partial**
  - System should be still usable, maybe at a reduced capacity
- Probability of failure in a WSC is very high, as it grows with number of components
- A **fault tolerant system** is able to meet its specifications even with partial failures

Fault tolerance: requirements

- Fault tolerance determined by:

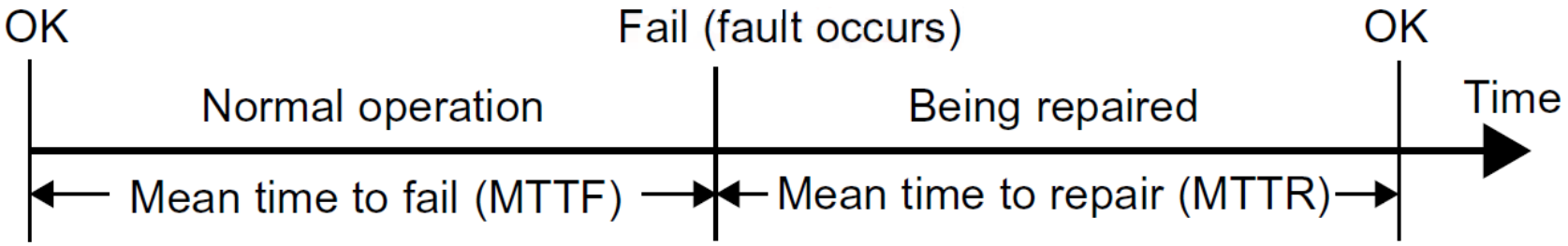
A. Reliability: continuity of service delivery
  - How long a system can operate without failure

B. Availability: readiness for usage
  - Percentage of time that a system is available to the user

C. Maintainability: how easily can a failed system be repaired

Fault tolerance: metrics

- **Mean Time To Failure (MTTF):** average time until a component fails
  - Reliability is measured by MTTF
- **Mean Time To Repair (MTTR):** average time needed to repair a component
  - Maintainability is measured by MTTR
- Availability = MTTF / (MTTF + MTTR)



Fault tolerance

- How do we increase process resilience?

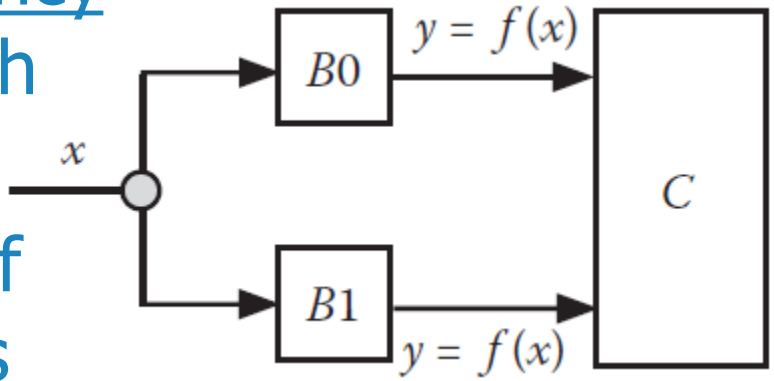
1. Increase MTTF
   - More reliable hardware (higher cost)
   - Bug-free software (feasible?)
2. Decrease MTTR
   a) Redundancy: replicate HW and SW components to mask completely failures to the application
   b) Recovery: failures may temporarily affect the application but correct execution is eventually restored

Contents

- Introduction
- Process scheduling
- **Process resilience**
  - Introduction
  - **Redundancy, state machines, and quorums**
  - Recovery strategies
- Virtualization

Modular redundancy

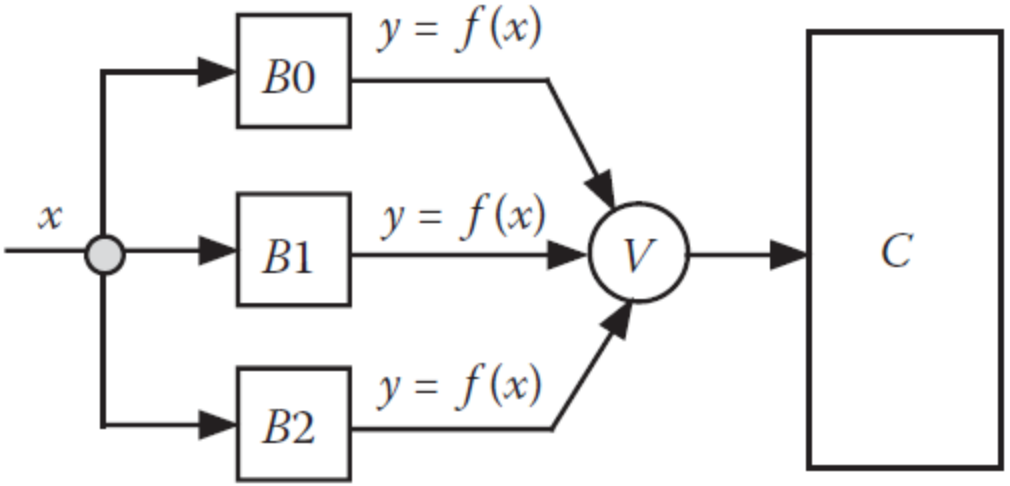
- Replicate processes or modules so that the correct ones will make up for those that fail
- Double modular redundancy can tolerate a single crash or omission failure
- With **f+1** replicas, up to f crash or omission failures can be tolerated



- Can detect (not tolerate) Byzantine failures by comparing the output of each replica

Modular redundancy

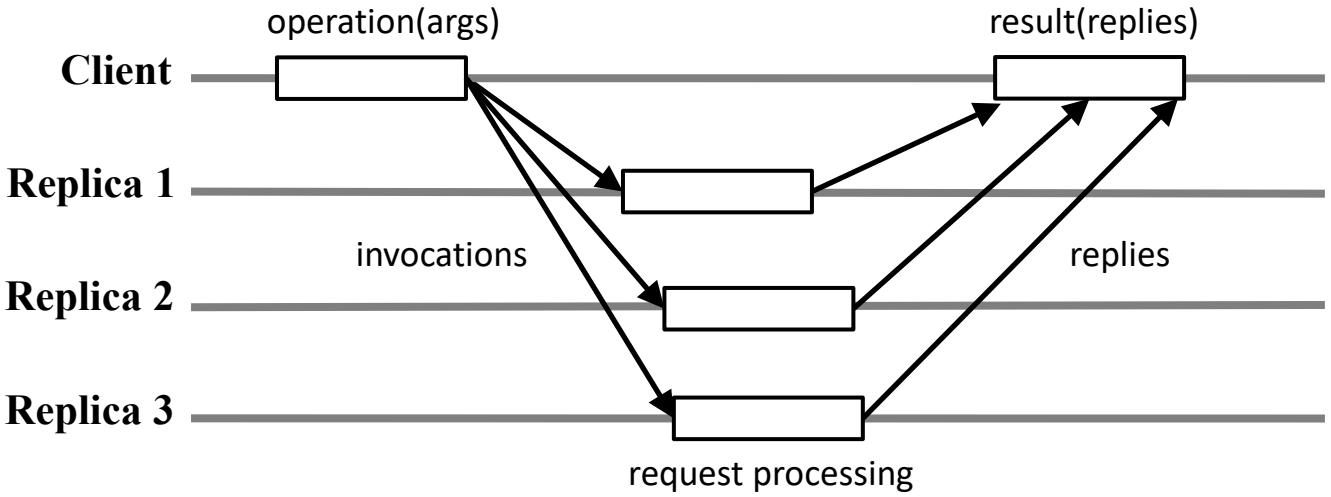
- Triple modular redundancy can tolerate a single Byzantine failure
  - Output is decided by a voter through **majority**
- With **2f+1** replicas, up to f Byzantine failures can be tolerated



Courtesy of [Ghos14]

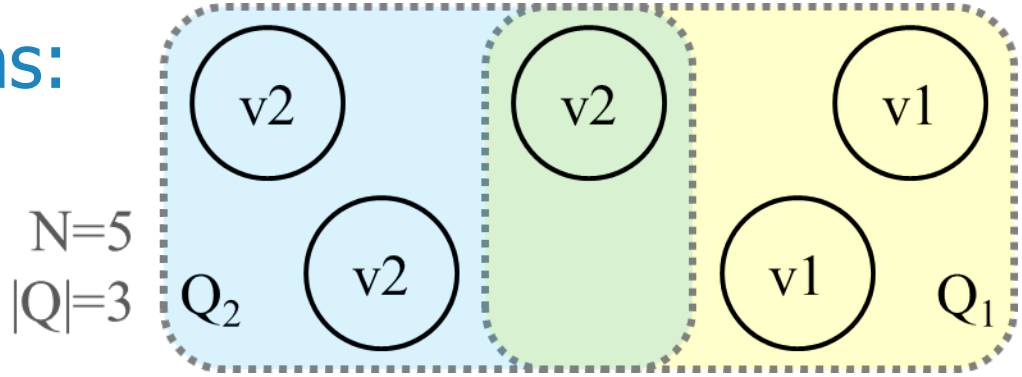
State machine replication (SMR)

- Modular redundancy is the basis for SMR
  - Client sends requests to ALL the process replicas
  - Replicas are deterministic: their states and outputs are identical if they run the same commands
  - Client awaits 1 (crash) / f+1 identical replies (Byz)



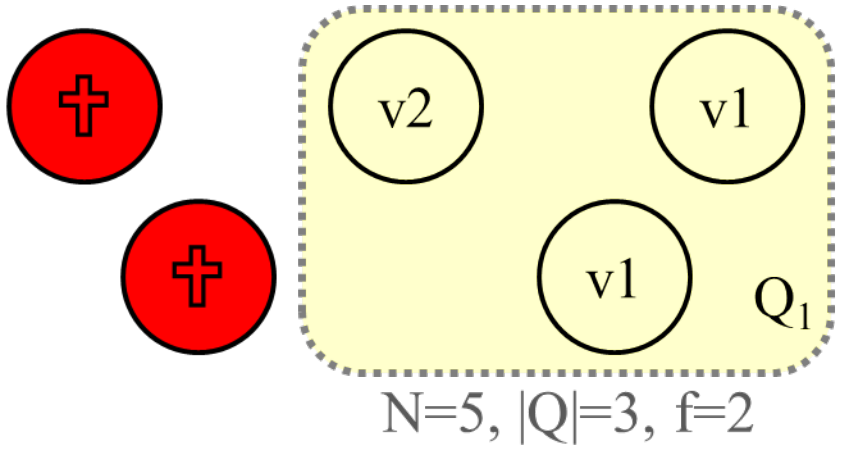
Quorum systems

- Collection of subsets of processes (quorums), each pair of which intersect: $Q_i \cap Q_j \neq \emptyset$
- A quorum can represent the entire system
  - Improves SMR availability and performance (client only needs to contact nodes in a quorum)
  - Thanks to the intersection property, updates in a quorum can be made consistent in others
- Majority quorums: $|Q| = \lfloor N/2 \rfloor + 1$

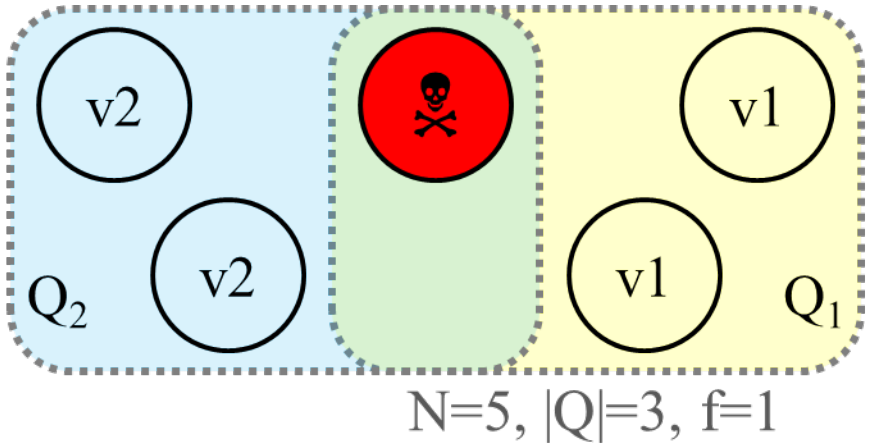


Quorum systems

- **2f+1** processes can tolerate f crash or omission failures

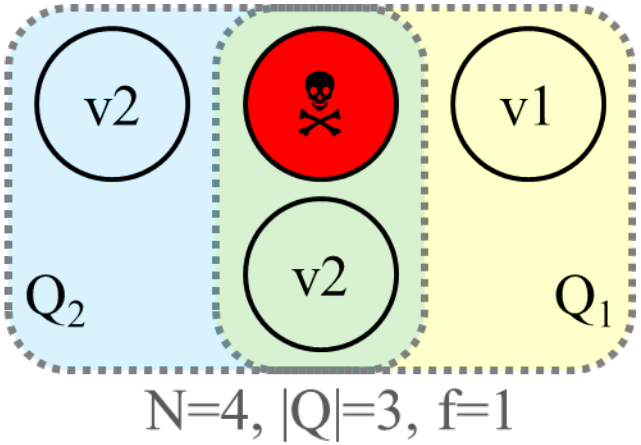


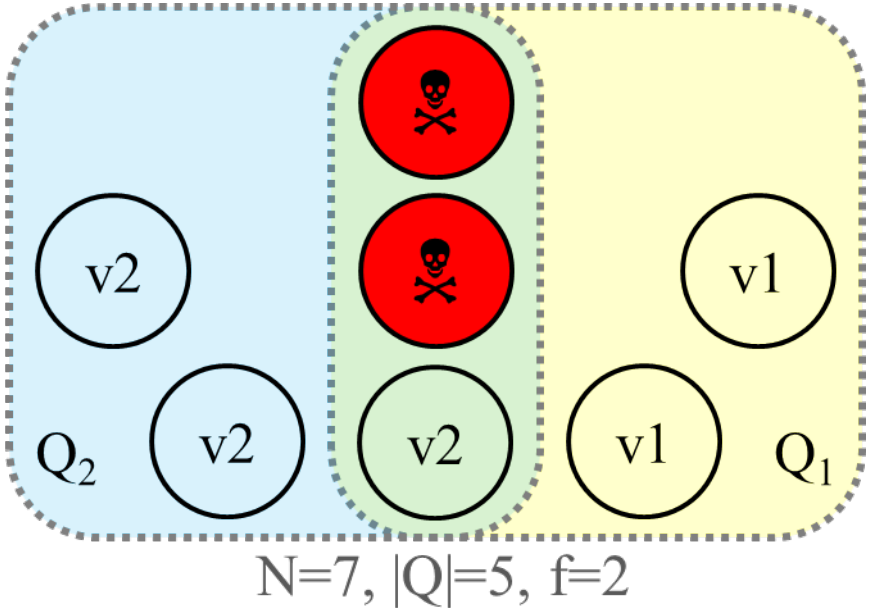
- With Byzantine faults, we need bigger quorum intersection to mask responses from faulty processes



Quorum systems

A. Any two quorums intersect in at least **f+1** processes, and thus in **1** correct process
   - **3f+1** processes can tolerate f Byzantine failures
   - $|Q| = \lfloor (N + f)/2 \rfloor + 1$

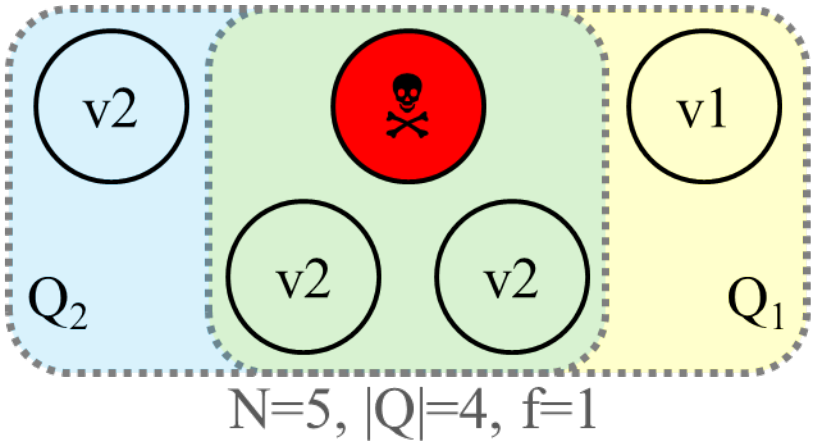


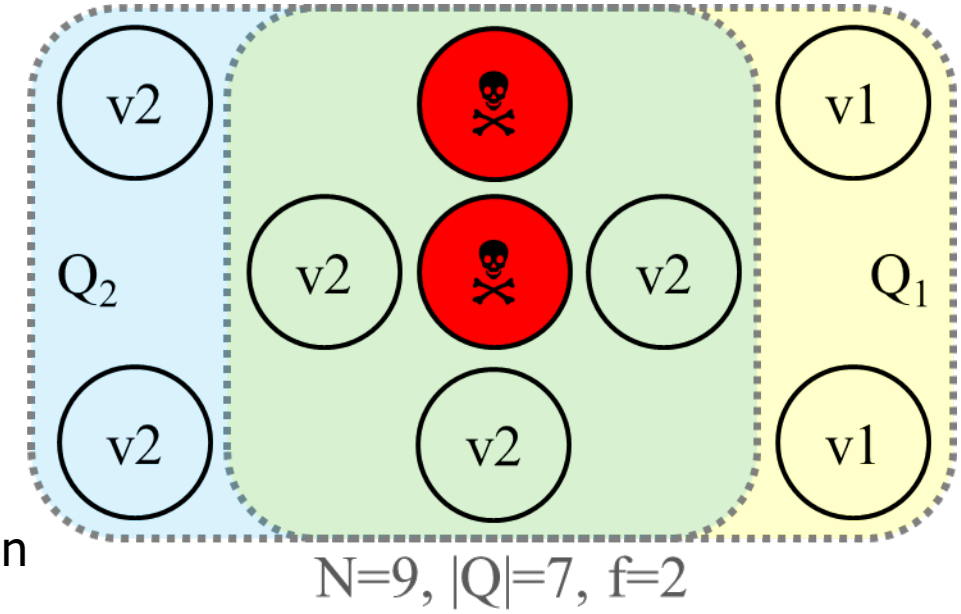


Requires self-verifying data (with public/private keys) so that Byzantine values can be discarded. Read returns the most recent **verifiable** version

Quorum systems

B. Any two quorums intersect in at least **2f+1** processes, and thus in **f+1** correct ones
   - **4f+1** processes can tolerate f Byzantine failures
   - $|Q| = \lfloor (N + 2f)/2 \rfloor + 1$





Read returns the most recent version with at least **f+1 representatives**

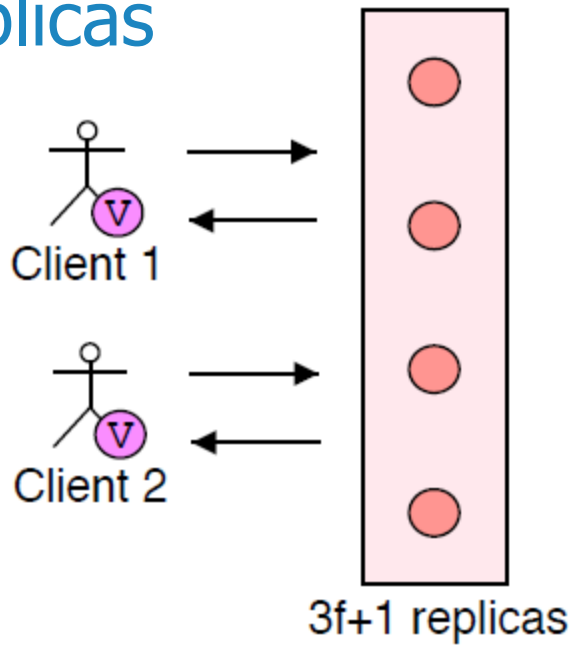
State machine replication (SMR)

- Previous solutions fall short when there are:
  - Concurrent writes from different clients
  - Complex operations beyond reads and writes
    - e.g., read-modify-write operations
  - Byzantine-faulty clients
- In these cases, replicas need to **agree** on the sequence of commands and their order
  - Need for **consensus**, e.g. Paxos, Raft, BFT
- Consensus algorithms are presented in the 'Distributed Systems' module of CPDS course

State machine replication (SMR)

- Consensus is possible only if there is a minimum number of replicas
  - 2f+1 replicas to tolerate f crash or omission failures †
  - 3f+1 replicas to tolerate f Byzantine failures †

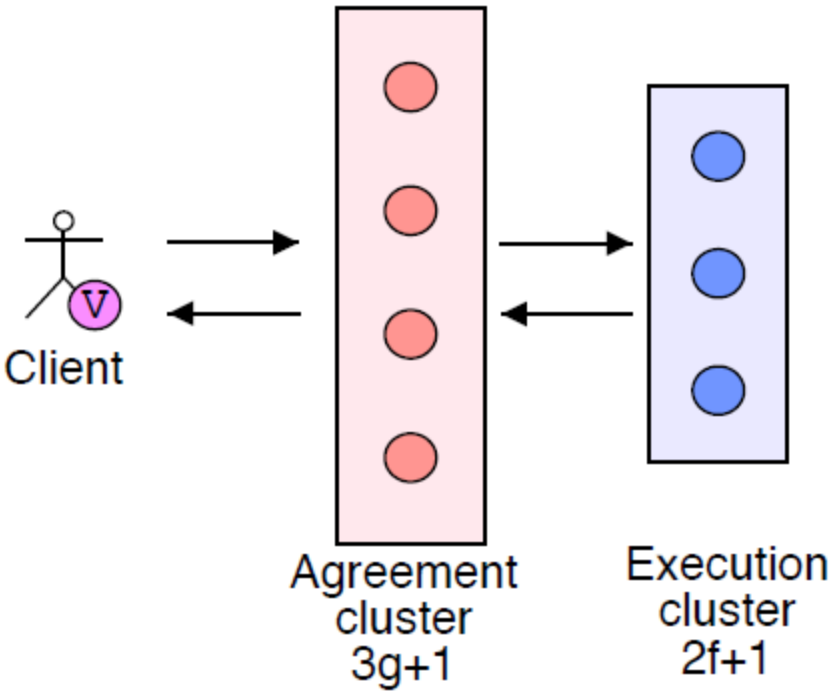
† Assuming asynchronous systems



- This is higher than the replicas needed by modular redundancy to run service correctly

State machine replication (SMR)

- Idea: separate the agreement on the order of commands from their execution on the state machine replicas to reduce their number



Failure mode	Agreement replicas	Execution replicas
Crash	2g+1	f+1
Byzantine	3g+1	2f+1

Agreement nodes can be part of a generic middleware service that may be **reused** across applications
⇒ coordination service

Contents

- Introduction
- Process scheduling
- **Process resilience**
  - Introduction
  - Redundancy, state machines, and quorums
  - **Recovery strategies**
- Virtualization

Recovery strategies

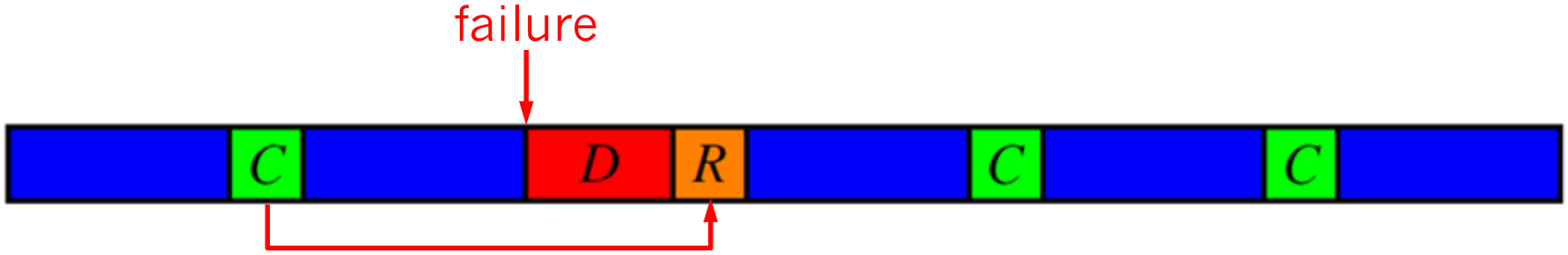
- Once a failure has occurred, it is essential that the system **recovers** to a correct state
  - Replace erroneous state with error-free state
- Two main forms of recovery:
  A. **Forward recovery:** Find a new error-free state from which the system can continue operation
  B. **Backward (or rollback) recovery:** Bring the system back into a previous error-free state
- Backward recovery is more common than forward recovery in practice

Backward recovery

1. Record the system state periodically onto stable storage to establish **recovery points**

   ⇒ Make **C**heckpoints (a.k.a. snapshots)

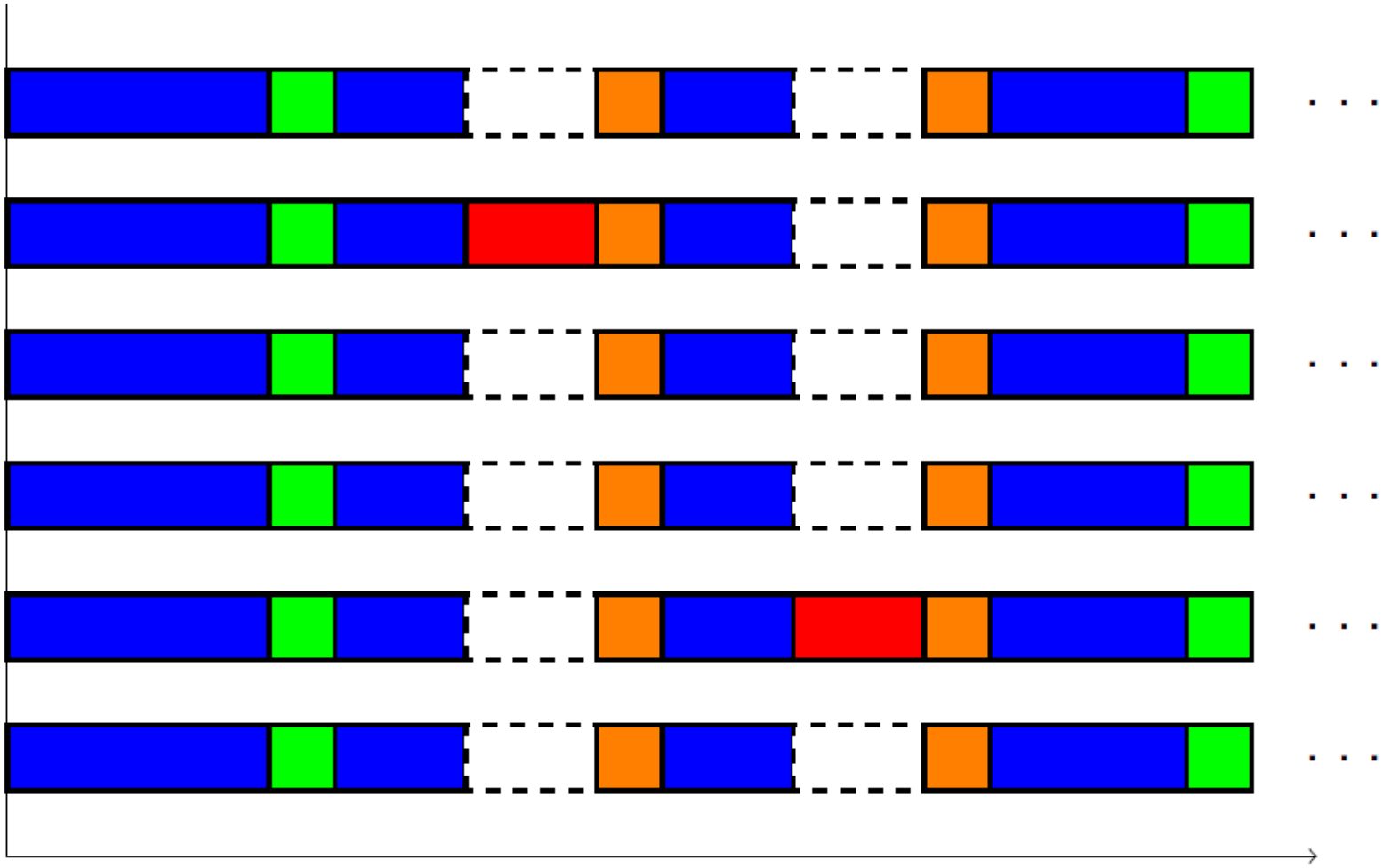
2. Upon a failure, **D**owntime to reboot or bring a spare host online
3. **R**ecover the system by rolling back to the previous checkpoint



Recovery of distributed programs

- Checkpoint/Restore of sequential programs is straightforward $\Rightarrow$ there is a single state
- But the state of a distributed program is:
  i. The individual states of all the processes
  ii. The states of the communication channels
- Upon a failure, inter-process dependencies may force some processes that did not fail to roll back as well to avoid orphan messages
  - If a process P has recorded the receipt of a message, there should also be a process Q that has recorded the sending of that message

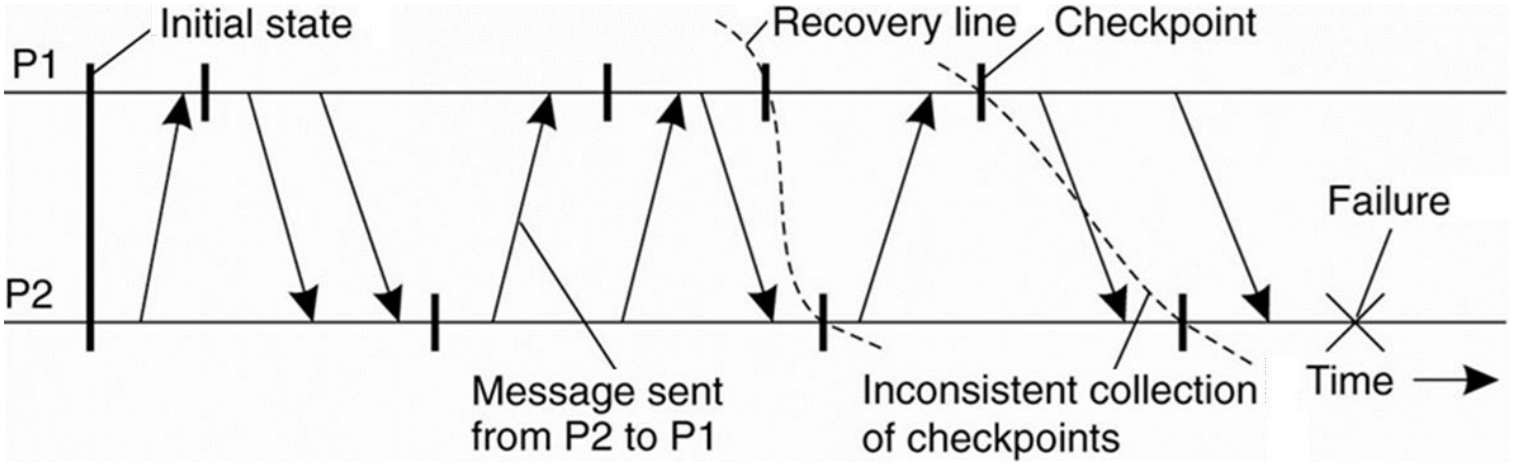
Recovery of distributed programs



Courtesy of [Casa17]

Global consistent recovery state

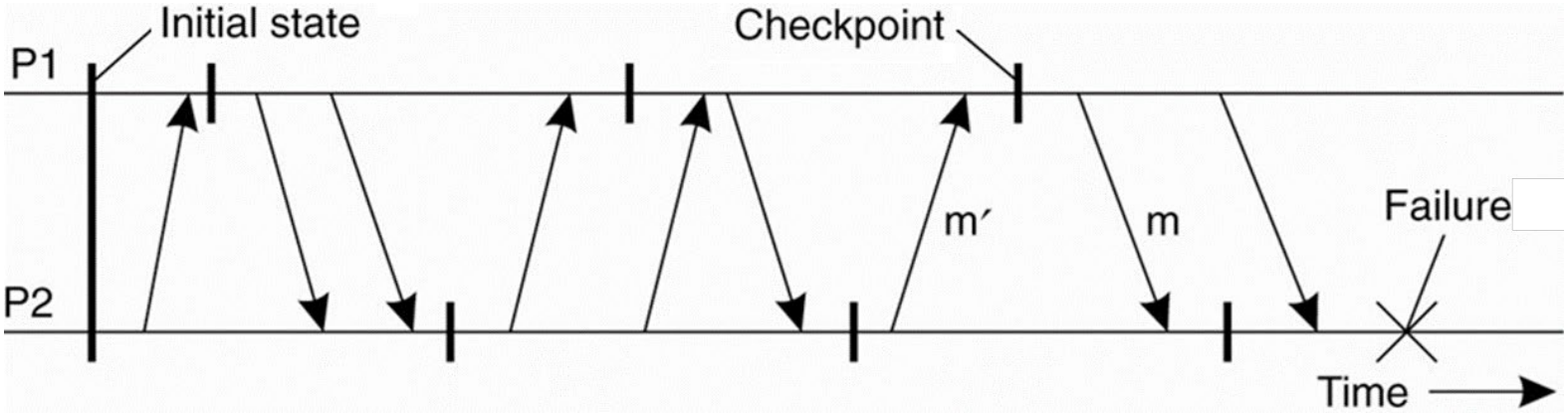
- Processes must identify a **recovery line** from where to recover ⇒ the most recent **consistent** collection of checkpoints (without orphans)
- There are mainly two techniques for that:
  1. Uncoordinated (or independent)
  2. Coordinated (or consistent)



Courtesy of [TaSt07]

Independent checkpointing

- Each process checkpoints independently of other processes from time to time
- Upon a failure, roll back each process to its most recently saved state and check if those local states jointly form a recovery line (consistent global state)
  - Processes need to keep rolling back until a recovery line is found (possible domino effect)



Courtesy of [TaSt07]

Independent checkpointing

- ↑ Take a checkpoint when it is most convenient
- ↓ Requires global coordination to compute the recovery line ⇒ recovery is complex
- ↓ Potential domino effect, which may cause the loss of a large amount of useful work
- ↓ Possible useless checkpoints that will never be part of a global consistent state
- ↓ Each process must keep multiple checkpoints

Coordinated checkpointing

- Each process checkpoints after global coordination

A. Blocking coordination
   1. A coordinator multicasts a *checkpoint-request* message
   2. A participant receiving such a message freezes execution, takes a checkpoint, queues any subsequent incoming messages, and acknowledges to the coordinator
   3. After getting all the ACKs, the coordinator broadcasts a *checkpoint-done* message to resume all the processes

   ↓ Checkpointing freezes the ongoing execution

B. Non-blocking coordination

   ↑ Checkpointing while processes continue their execution

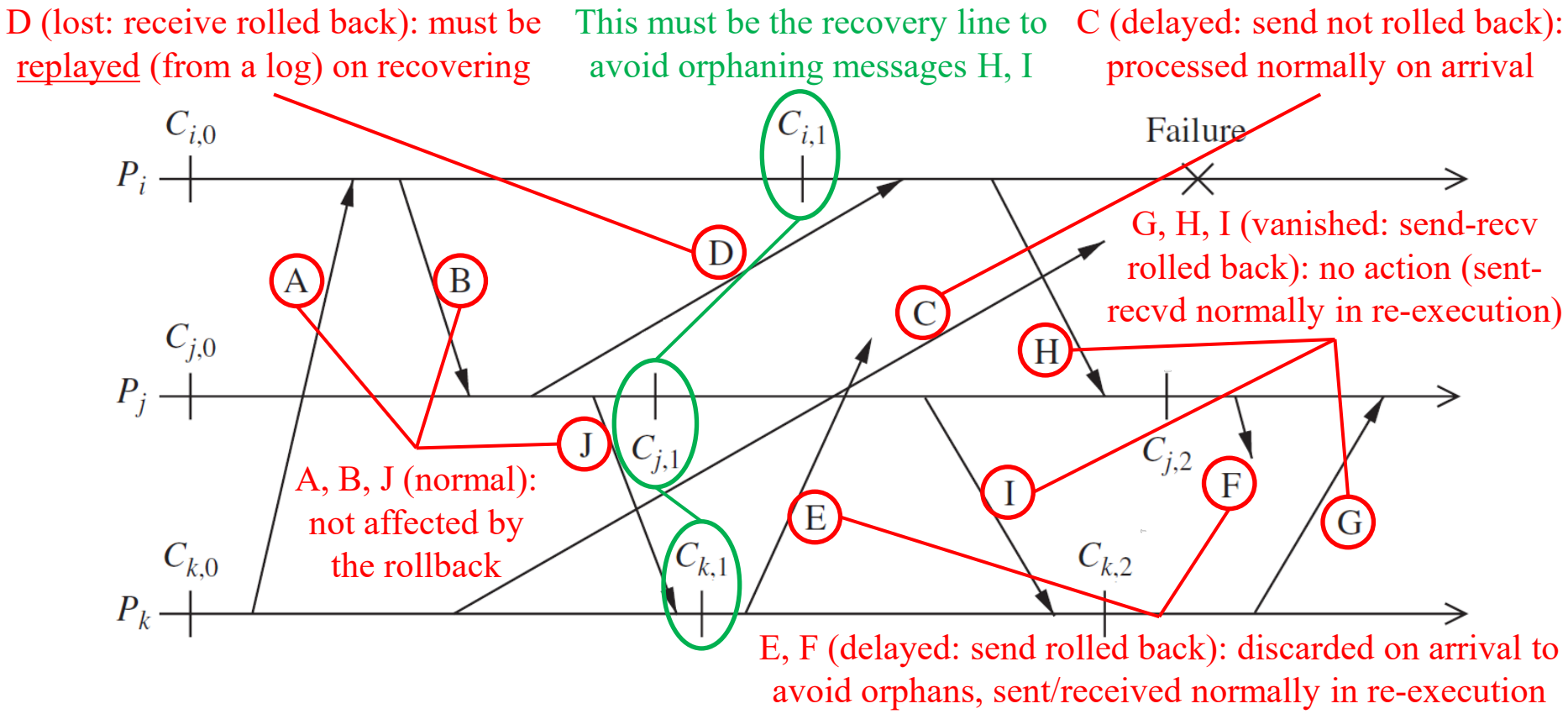
   – e.g. Chandy & Lamport's Snapshot Algorithm
     - Presented in the 'Distributed Systems' module of CPDS course

Coordinated checkpointing

- ↑ Saved state is automatically globally consistent
- ↑ Not susceptible to the domino effect, since every process always restarts from its most recent checkpoint
- ↑ Each process has to maintain only one permanent checkpoint on stable storage
- ↓ Large latency involved in taking a checkpoint
  - Critical factor with blocking coordination

Rollback recovery: Message handling

- Rollback recovery needs consistent states but also proper message handling (esp. with independent checkpointing)



Courtesy of [KsSi11]

Backward recovery

- Checkpoint/Restore is an effective technique for recovering failures, but ...
  - ↓ Making checkpoints is expensive
  - ↓ Some states can never be rolled back to

A. Failure prediction

- 'Precursor' events can provide a good sense of when a PE might fail in the future

a) Checkpoint proactively only when a failure seems likely in the short term

b) Migrate all processes running in a PE when a failure seems likely in the short term

Backward recovery

B. Log-based recovery: Combine checkpoints with message logging

1. Processes take infrequent checkpoints
2. Processes log all the received messages onto local stable storage (pessimistic)
3. Upon a failure, the crashed process rolls back to its latest checkpoint and replays the messages received since that checkpoint to itself (in the right order)

↑ If a process fails, no other process will need to be rolled back to ensure consistency

References

[ArAr23] Arpaci-Dusseau, R.H.; Arpaci-Dusseau, A.C.; Operating Systems: Three Easy Pieces, v.1.10, Arpaci-Dusseau Books, 2023

[TaBo23] A.S. Tanenbaum and H. Bos. 2023. Modern Operating Systems, 5th ed. Pearson Education, Inc., Hoboken, NJ, USA

[TaSt07] A.S. Tanenbaum and M. van Steen. 2007. Distributed Systems: Principles and Paradigms, 2nd edition Prentice-Hall, Inc., Upper Saddle River, USA

[Casa17] H. Casanova, ICS 632 Principles of High Performance Computing, 2017

[Feit97] D.G. Feitelson, Job scheduling in multiprogrammed parallel systems, IBM Research Report RC 19790 (87657), October 1994 (revised in August 1997)

[FeRS04] D.G. Feitelson, L. Rudolph, and U. Schwiegelshohn. 2004. Parallel job scheduling — a status report. In 10th International Conference on Job Scheduling Strategies for Parallel Processing (JSSPP'04), Springer-Verlag, pp. 1-16

[Brec94] Timothy Benedict Brecht, Multiprogrammed Parallel Application Scheduling in NUMA Multiprocessors, Technical Report CSRI-303, June 1994

[YoDa02] B. S. Yoo and C. R. Das, A fast and efficient processor allocation scheme for mesh-connected multicomputers, in IEEE Transactions on Computers, vol. 51, no. 1, pp. 46-60, Jan. 2002

References

[Feit01] D.G. Feitelson. Metrics for Parallel Job Scheduling and Their Convergence. In 7th International Workshop on Job Scheduling Strategies for Parallel Processing (JSSPP'01), Springer-Verlag, pp. 188-206, 2001

[HwDF11] K. Hwang, J. Dongarra, G. Fox; Distributed and Cloud Computing, Morgan Kaufmann, 2011

[Schw16] Malte Schwarzkopf, The evolution of cluster scheduler architectures, Cambridge Systems at Scale (CamSaS) Blog, 2016, https://www.cl.cam.ac.uk/research/srg/netos/camsas/blog/2016-03-09-scheduler-architectures.html

[Ghos14] S. Ghosh. Distributed Systems: An Algorithmic Approach, 2nd Edition, Chapman and Hall/CRC, 2014

[YMVA03] J. Yin, J-P. Martin, A. Venkataramani, L. Alvisi, and M. Dahlin. 2003. Separating agreement from execution for byzantine fault tolerant services. 19th ACM symposium on Operating systems principles (SOSP'03), pp. 253-267

[RoHL07] C.A.F. De Rose, H.-U. Heiss, B. Linnert, Distributed dynamic processor allocation for multicomputers, Parallel Computing, Vol. 33, no. 3, 2007, pp. 145-158

[KsSi11] A.D. Kshemkalyani, M. Singhal, Distributed Computing: Principles, Algorithms, and Systems, Cambridge University Press, March 2011